

19-apr-24

oracle :-

1 sql

2 pl/sql

Database :-

=> a DB is a organized collection of interrelated data. For example
a univ db stores data related to students,courses,faculty etc and
a bank db stores data related to cust,accounts,loans,employees etc

univ db	bank db
students	customers
courses	accounts
faculty	loans
colleges	employees

Types of Databases :-

1 OLTP DB (online transaction processing)

2 OLAP DB (online analytical processing)

=> organizations uses OLTP DB for storing day-to-day transactions and OLAP for analysis.

=> OLTP is for running business and OLAP is for to analyze business.

=> day-to-day operations on db includes

C	create
R	read
U	update
D	delete

DBMS :-

=> DBMS stands for Database Management System

=> It is a software used to manage database.

=> DBMS allows users to create,read,update,delete data.

=> DBMS is an inteface between user and database.

USER-----DBMS-----DB

Evolution of DBMS :-

1960	FMS (File Mgmt System)
1970	HDBMS (Hierarchical DBMS) NDBMS (Network DBMS)
1980	RDBMS (Relational DBMS)
1990	ORDBMS (Object Relational DBMS)

Types of DBMS :-

HDBMS
NDBMS
RDBMS
ORDBMS

RDBMS :-

=> Relational model concepts are introduced by E.F.CODD
=> E.F.CODD introduced 12 rules called codd rules
=> a dbms said to be perfect rdbms if supports all 12 rules

Information rule :-

=> according to information rule data must be organized in tables i.e. rows and columns

CUST

CID NAME ADDR => COLUMNS / FIELDS / ATTRIBUTES

10 A HYD

11 B BLR => ROW / RECORD / TUPLE

DATABASE = COLLECTION OF TABLES

TABLE = COLLECTION OF ROWS & COLS

ROW = COLLECTION OF FIELD VALUES

COLUMN = COLLECTION OF VALUES ASSIGNED TO ONE FIELD

=> every table must contain primary key to uniquely identify the records

ex :- accno,empid,aadharno,panno,voterid

RDBMS features :-

- 1 easy to access and manipulate data
- 2 less redundancy (duplication of data)
- 3 more security
- 4 gurantees data quality
- 5 supports data sharing
- 6 supports transactions

RDBMS softwares :- (SQL Databases)

ORACLE	from oracle corp
MYSQL	from oracle corp
SQL SERVER	from microsoft
POSTGRESQL	from postgresql
RDS	from amzon

NoSQL Databases :-

MongoDB
cassandra

20-apr-24

ORDBMS :-

=> ordbms is combination of rdbms & oops

ordbms = rdbms + oops (reusability)

=> rdbms doesn't support reusability but ordbms supports reusability.

=> ordbms supports reusability by using UDT (user define type).

examples :-

oracle upto 7 ver rdbms
oracle from 8 ver ordbms
sql server
postgresql

=====

ORACLE =====

=> oracle is basically rdbms software and also supports features of ordbms and used to manage database.

versions of oracle :-

History :-

1977 SDL (software development laboratories)
1979 RSL (Relational software laboratories)
1983 oracle systems corporation
1995 oracle corporation

versions :-

2,3,4,5,6,7,8i,9i,10g,11g,12c,18c,19c,21c,23c

i => internet
g => grid
c => cloud

=> from version 8 onwards suffix "i" is added because from 8 onwards oracle supports internet applications i.e. oracle db can be used for web applications.

=> grid means collection of servers, from 10g onwards oracle db can be accessed through multiple servers and advantage of grid is if one server is down we can access db through another server.

=> from 12c onwards oracle db can be deployed in

- 1 on premises server
- 2 cloud server

=> " on premises " db is deployed in the server managed by client.

=> " on cloud " db is deployed in the server managed by cloud service provider
cloud reduces initial investment for client.

CLIENT / SERVER Architecture :-

- 1 server
- 2 client

server :-

=> sever is a system where oracle is installed and running

=> inside the server oracle manages

- 1 DB
- 2 INSTANCE

=> DB is created in hard disk and acts as permanent storage

=> INSTANCE is created in ram and acts as temporary storage

CLIENT :-

=> a client is also a system from where users can

- 1 connects to server
- 2 submit requests to server
- 3 receives response from server

client tool :-

SQLPLUS / SQL DEVELOPER
JDBC

USER----SQLPLUS-----ORACLE-----DB

USER----JDBC-----ORACLE-----DB

SQL :-

=> SQL stands for structured query language

=> It is a language used to communicate with oracle

=> user communicates with oracle by sending commands called queries

=> a query is a command / instruction / question submitted to oracle to perform some operation over db.

=> sql is introduced by IBM and initial name was "SEQUEL" and later it is renamed to SQL.

=> sql is common to all relational databases

ORACLE	MYSQL		POSTGRESQL	SQL SERVER
SQL	SQL	SQL	SQL	

USER----SQLPLUS-----SQL-----ORACLE-----DB

USER---MYSQLWORKBENCH-----SQL-----MYSQL-----DB

USER----SSMS-----SQL-----SQL SERVER-----DB

=> based on operations over db sql is categorized into following sublanguages

- DDL (Data Definition Lang)
- DML (Data Manipulation Lang)
- DQL (Data Query Lang)
- TCL (Transaction Control Lang)
- DCL (Data Control Lang)

SQL

DDL DML DQL TCL DCL

create	insert	select	commit	grant
alter	update		rollback	revoke
drop	delete		savepoint	
truncate		merge		

rename insert all
flashback
purge

SCHEMA :-

=> a user in oracle db is called schema

SERVER
 DATABASE
 USER
 TABLE
 DATA

SERVER
 XE
 SYS / MANAGER (admin)
 SYSTEM / MANAGER (admin)

How to connect to oracle :-

=> to connect to oracle open sqlplus and enter username & password

USERNAME :- SYSTEM
PASSWORD :- MANAGER

OR

USERNAME :- SYSTEM/MANAGER

CREATING USER IN ORACLE DB :-

=> only DBA are having permissions to create new user

STEP 1 :- connect as DBA

USERNAME :- SYSTEM / MANAGER

STEP 2 :- create user

syntax :-

```
CREATE USER <NAME> IDENTIFIED BY <PWD>
DEFAULT TABLESPACE USERS
QUOTA UNLIMITED ON USERS ;
```

Example :-

```
SQL>CREATE USER BATCH32 IDENTIFIED BY ORACLE
      DEFAULT TABLESPACE USERS
      QUOTA UNLIMITED ON USERS ;
```

STEP 3 :- Granting permissions to user

```
SQL>GRANT CONNECT,RESOURCE TO BATCH32 ;
```

CONNECT => to connect to db
RESOURCE => to create tables
DBA => all permissions

if connec,resource are granted then user is called normal user
if dba granted then user is called admin user

Datatypes in ORACLE :-

=> a datatype specifies what type of data allowed in a column
=> based on datatype oracle allocates memory

DATATYPES

CHAR		NUMERIC	DATE	BINARY
ASCII	UNICODE	number(p) number(p,s)	date timestamp	bfile blob
char	nchar			
varchar2	nvarchar2			
long	nclob			
clob				

CHAR(size) :-

=> allows character data upto 2000 chars

=> recommended for fixed length char columns

ex :- NAME CHAR(10)

sachin----

wasted

ravi-----

wasted

=> in CHAR datatype extra bytes are wasted , so don't use char for variable length fields and use char for fixed length fields

ex :- GENDER CHAR(1)

M

F

STATE_CODE CHAR(2)

AP

TG

COUNTRY_CODE CHAR(3)

IND

USA

VARCHAR2(size) :-

=> allows character data upto 4000 chars

=> recommended for variable length fields

ex :- NAME VARCHAR2(10)

sachin----

released

ravi-----
released

=> char / varchar2 allows ascii chars (256) that includes A-Z,a-z,0-9,special chars
i.e. char / varchar2 allows alphanumeric data.

ex :- PANNO CHAR(10)

ASKD-----

VEHNO CHAR(10)

TS09AB1234

EMAILID VARCHAR2(20)

abc123@gmail.com

LONG :-

=> allows character data upto 2GB

ex :- review LONG

CLOB :- (character large object)

=> allows character data upto 4GB

ex :- TEXT CLOB

NCHAR/NVARCHAR2/NCLOB :- (N => National)

=> allows unicode chars (65536) that allows all ascii and also chars belongs to
different languages.

=> a unicode char occupies 2 bytes

NUMBER(P) :-

=> allows numbers without decimal (integers) upto 38 digits

EX :- EMPID NUMBER(4)

10

100

1000

10000 => NOT ALLOWED

AADHARNO NUMBER(12)

PHONE NUMBER(10)

ACCNO NUMBER(12)

NUMBER(P,S) :-

=> allows numbers with decimal

P => precision => total no of digits allowed

S => scale => no of digits allowed after decimal

ex :- SALARY NUMBER(7,2)

5000

5000.55

50000.55

500000.55 => not allowed

5000.5678 => allowed => 5000.57

5000.5634 => allowed => 5000.56

NOTE :- if before decimal exceeds number is not accepted
if after decimal exceeds number is accepted and rounded

BALANCE NUMBER(11,2)

SAVG NUMBER(5,2)

DATE :-

=> allows date & time

=> time is optional

=> if not entered oracle stores 12:00AM
=> default date format in oracle DD-MON-YY / YYYY

ex :- DOB DATE

23-APR-03 => 23-APR-2003 00:00:00
15-OCT-98 => 15-OCT-2098 00:00:00
15-OCT-1998 => 15-OCT-1998 00:00:00

=> date occupies 7 bytes => DD MM YYYY HH MI SS
 1 1 2 1 1 1

TIMESTAMP :-

=> allows date,time and also milliseconds,timezone

ex :- T TIMESTAMP

23-APR-24	12:30:20.123	+5:30
-----	-----	-----
DATE	TIME	MS TZ

=> timestamp occupies 9 to 13 bytes depends on milliseconds

Binary Types :-

=> Binary types allows multimedia objects like audio,video,images

- 1 BFILE (Binary File)
- 2 BLOB (Binary Large Object)

=> bfile is called external lob because lob stored outside db and db stores path
=> blob is called internal lob because lob stored inside db

24-apr-24

CREATING TABLES IN ORACLE DB :-

CREATE TABLE <TABNAME>
(

```
COLNAME  DATATYPE(SIZE) ,  
COLNAME  DATATYPE(SIZE),  
-----
```

);

Rules :-

- 1 name should start with alphabet
- 2 name should not contain spaces & special chars but allows _ \$ #
- 3 name can be upto 128 chars
- 4 table can have 1000 cols
- 5 no of rows unlimited

```
ex :-  123EMP      => INVALID  
        EMP 123    => INVALID  
        EMP*123    => INVALID  
        EMP_123    => VALID
```

Exampe :-

=> create table with following structure ?

```
EMP  
EMPID      ENAME  JOB   SAL   HIREDATE  DNO
```

```
CREATE TABLE EMP  
(  
  EMPID  NUMBER(4),  
  ENAME  VARCHAR2(10),  
  JOB    VARCHAR2(10),  
  SAL    NUMBER(7,2),  
  HIREDATE  DATE ,  
  DNO    NUMBER(2)  
);
```

USER---SQLPLUS-----CREATE TABLE-----ORACLE-----DB

=> above command created table structure / definition / metadata that includes
columns,datatype and size.

DESC :- (DESCRIBE)

=> command used to see the structure of the table

syn :- DESC <TABNAME>

Ex :-

SQL>DESC EMP

EMPID	NUMBER(4)
ENAME	VARCHAR2(10)
JOB	VARCHAR2(10)
SAL	NUMBER(7,2)
HIREDATE	DATE
DNO	NUMBER(2)

Question :-

CUST
CUSTID NAME GENDER DOB AADHARNO PANNO ADDR CITY STATE PHONE
PHOTO EMAILID

INSERTING DATA INTO TABLE :-

=> "INSERT" command is used to insert data into table

=> we can insert

- 1 single row
- 2 multiple rows

insertng single row :-

syn :- INSERT INTO <TABNAME> VALUES(V1,V2,V3,-----);

Ex :-

SQL>INSERT INTO EMP VALUES(100,'SACHIN','CLERK',4000,'24-APR-24',20);
SQL>INSERT INTO EMP VALUES(101,'ARVIND','ANALYST',8000,SYSDATE,30);

NOTE :-

=> above insert commands inserted data into instance (ram) to save this data execute COMMIT.

```
SQL>COMMIT;
```

Inserting multiple rows :-

=> insert command can be executed multiple times with different values by using variables prefixed with "&".

```
SQL>INSERT INTO EMP VALUES(&EMPNO,&ENAME,&JOB,&SAL,&HIREDATE,&DNO);
```

Enter value for empno: 102

Enter value for ename: 'RAHUL'

Enter value for job: 'MANAGER'

Enter value for sal: 6000

Enter value for hiredate: '15-MAR-20'

Enter value for dno: 10

```
SQL> / (executes previous command)
```

Enter value for empno: 103

Enter value for ename: 'KUMAR'

Enter value for job: 'ANALYST'

Enter value for sal: 7000

Enter value for hiredate: '10-OCT-18'

Enter value for dno: 20

```
SQL> /
```

Enter value for empno: 104

Enter value for ename: 'SATISH'

Enter value for job: 'CLERK'

Enter value for sal: 5000

Enter value for hiredate: '18-NOV-21'

Enter value for dno: 30

```
SQL>COMMIT ;
```

NOTE :-

=> in normal exit operations are saved and in abnormal exit operations are cancelled

SQL>EXIT ; => normal exit

closing sqlplus window => abnormal exit

25-APR-24

INSERTING NULLS :-

=> a NULL means blank or empty

=> it is not equal to 0 or space

=> NULLs can be inserted in two ways

method 1 :-

```
INSERT INTO EMP VALUES(106,'RAVI',",",3000,SYSDATE,NULL);
```

method 2 :-

```
INSERT INTO EMP(EMPID,ENAME,SAL,HIREDATE)
VALUES (107,'DAVID',4000,'18-SEP-22') ;
```

remaining two fields job,dno filled with nulls

Operators in oracle :-

Arithmetic Operators	=>	+	-	*	/			
Relational Operators =>	>	>=	<	<=	=	<>	or	!=
Logical Operators	=>	AND	OR	NOT				
Special Operators	=>	BETWEEN						
		IN						
		LIKE						
		IS						
		ANY						
		ALL						
		EXISTS						
Set Operators	=>	UNION						
		UNION ALL						
		INTERSECT						
		MINUS						

Displaying Data :-

=> "SELECT" command is used to display data from table.

=> we can display all rows and all cols

=> we can display specific rows and specific cols

syn :- `SELECT columns / * FROM tablename ;`

SQL = ENGLISH
QUERIES = SENTENCES
CLAUSES = WORDS

Ex :-

=> display employee names and salaries ?

`SELECT ENAME,SAL FROM EMP ;`

=> display employee names,jobs and hiredate ?

`SELECT ENAME,JOB,HIREDATE FROM EMP ;`

=> display all the data from emp table ?

`SELECT * FROM EMP ;`

WHERE clause :-

=> WHERE clause is used to get specific row/rows from table based on a condition

`SELECT columns / *
FROM tablename
WHERE condition ;`

condition :-

`COLNAME OP VALUE`

=> condition is always a relational expression that returns TRUE / FALSE

=> OP must be any relational operator like > >= < <= = <>

=> if cond = true row is selected

=> if cond = false row is not selected

ex :-

=> display employee details whose id = 103 ?

```
SELECT * FROM EMP WHERE EMPID = 103;
```

=> display employee details name = kumar ?

```
SELECT * FROM EMP WHERE ENAME = 'KUMAR' ;
```

note :- in oracle string comparison is case sensitive i.e. uppercase and lowercase strings are not same.

=> employees earning more than 5000 ?

```
SELECT * FROM EMP WHERE SAL > 5000 ;
```

=> employees joined after 2020 ?

```
SELECT * FROM EMP WHERE HIREDATE > 2020 ; => ERROR
```

```
SELECT * FROM EMP WHERE HIREDATE > '31-DEC-20' ;
```

=> employees joined before 2020 ?

```
SELECT * FROM EMP WHERE HIREDATE < '01-JAN-20' ;
```

NOTE :-

=> relational operators can be used with numeric,char and date fields

```
SELECT * FROM EMP WHERE ENAME > 'K' ;
```

```
SELECT * FROM EMP WHERE ENAME < 'K' ;
```

26-APR-24

compound condition :-

=> multiple conditions combined with AND / OR operators are called compound condition.

WHERE	COND1	AND	COND2	RESULT
	T	T	T	
	T	F	F	
	F		T	F
	F		F	F

WHERE	COND1	OR	COND2	RESULT
	T	T	T	
	T	F	T	
	F	T	T	
	F	F	F	

Ex :-

=> display employees working as clerk,manager ?

```
SELECT *
FROM EMP
WHERE JOB='CLERK' OR JOB='MANAGER' ;
```

=> employee details whose id = 100,103,105 ?

```
SELECT *
FROM EMP
WHERE EMPID = 100 OR EMPID = 103 OR EMPID = 105 ;
```

=> employees working as clerk and earning more than 4000 ?

```
SELECT *
FROM EMP
WHERE JOB='CLERK' AND SAL > 4000 ;
```

=> employees earning more than 5000 and less than 10000 ?

```
SELECT *
FROM EMP
WHERE SAL > 5000 AND SAL < 10000 ;
```

=> employees joined in 2020 year ?

```
SELECT *
FROM EMP
WHERE HIREDATE >= '01-JAN-2020' AND HIREDATE <= '31-DEC-2020' ;
```

=>

```
STUDENT
SNO  SNAME    S1    S2    S3
1    A      80    90    70
2    B      30    60    50
```

list of students who are passed ?

```
SELECT * FROM STUDENT WHERE S1>=35 AND S2>=35 AND S3>=35 ;
```

list of students who are failed ?

```
SELECT * FROM STUDENT WHERE S1<35 OR S2<35 OR S3<35 ;
```

=> list of employees working as clerk,manager and earning more than 5000 ?

```
SELECT *
FROM EMP
WHERE JOB='CLERK'
      OR
      JOB='MANAGER'
      AND
      SAL > 5000 ;
```

above query returns clerk records earning less than 5000 because SAL > 5000 is applied only to MANAGERS but not applied to CLERK because operator AND has got more priority then operator OR , to overcome this problem use () .

```
SELECT *
FROM EMP
WHERE (
      JOB='CLERK'
      OR
      JOB='MANAGER'
    )
      AND
      SAL > 5000 ;
```

IN operator :-

=> use IN operator for list comparison

=> use IN operator for "=" comparison with multiple value

WHERE COLNAME = V1,V2,V3,----- => INVALID

WHERE COLNAME IN (V1,V2,V3,---) => VALID

Ex :-

=> employees working as clerk,manager ?

SELECT * FROM EMP WHERE JOB IN ('CLERK','MANAGER');

=> employees whose id = 100,103,105 ?

SELECT * FROM EMP WHERE EMPID IN (100,103,105);

=> employees not working for dept 10,20 ?

SELECT * FROM EMP WHERE DNO NOT IN (10,20) ;

BETWEEN operator :-

=> use BETWEEN operator for range comparison

WHERE COLNAME BETWEEN V1 AND V2 (COL >= V1 AND COL <= V2)

=> employees earning between 5000 and 10000 ?

SELECT * FROM EMP WHERE SAL BETWEEN 5000 AND 10000 ;

=> employees not joined in 2020 year ?

SELECT *
FROM EMP
WHERE HIREDATE NOT BETWEEN '01-JAN-20' AND '31-DEC-20' ;

=> employees working as clerk,manager and earning between 5000 and 10000

and not joined in 2024 and working for dept 10,20 ?

```
SELECT *
FROM EMP
WHERE JOB IN ('CLERK','MANAGER')
      AND
      SAL BETWEEN 5000 AND 10000
      AND
      HIREDATE NOT BETWEEN '01-JAN-24' AND '31-DEC-24'
      AND
      DNO IN (10,20);
```

=> list of samsung,redmi,realme mobile phones price between 10000 and 20000 ?

```
PRODUCTS
prodid pname price  category      brand

SELECT *
FROM PRODUCTS
WHERE BRAND IN ('SAMSUNG','REDMI','REALME')
      AND
      PRICE BETWEEN 10000 AND 20000
      AND
      CATEGORY='MOBILES' ;
```

LIKE operator :-

=> use LIKE operator for pattern comparison

ex :- name starts with 's'
name ends with 'd'
name contains 'a'

```
WHERE COLNAME LIKE 'PATTERN'
```

=> pattern contains alphabets,digits,special chars and wildcard chars

WILDCARD chars :-

% => 0 or many chars

_ => exactly 1 char

=> employees name starts with 'S' ?

```
SELECT * FROM EMP WHERE ENAME LIKE 'S%';
```

=> employees name ends with 'D' ?

```
SELECT * FROM EMP WHERE ENAME LIKE '%D';
```

=> employees name contains 'A' ?

```
SELECT * FROM EMP WHERE ENAME LIKE '%A%';
```

=> employees where 'A' is the 4th char in their name ?

```
SELECT * FROM EMP WHERE ENAME LIKE '___A%';
```

=> 'A' is the 4th char from last ?

```
SELECT * FROM EMP WHERE ENAME LIKE '%A___';
```

=> name contains 5 chars ?

```
SELECT * FROM EMP WHERE ENAME LIKE '_____';
```

=> employees joined in OCT month ? (DD-MON-YY)

```
SELECT * FROM EMP WHERE HIREDATE LIKE '%OCT%';
```

=> joined in 2020 year ?

```
SELECT * FROM EMP WHERE HIREDATE LIKE '%20';
```

27-APR-24

IS operator :-

=> operator used to compare with NULL / NOT NULL

WHERE COLNAME IS NULL
WHERE COLNAME IS NOT NULL

=> list of employees not assigned to any dept ?

SELECT * FROM EMP WHERE DNO IS NULL ;

=> employees assigned to dept ?

SELECT * FROM EMP WHERE DNO IS NOT NULL ;

summary :-

WHERE COLNAME IN (V1,V2,V3,----)
WHERE COLNAME BETWEEN V1 AND V2
WHERE COLNAME LIKE 'PATTERN'
WHERE COLNAME IS NULL

ALIAS :-

=> alias means another name or alternative name

=> used to change column heading

COLNAME / EXPR [AS] ALIAS

Ex :-

=> display ENAME ANNSAL ?

SELECT ENAME,SAL*12 AS ANNSAL FROM EMP ;

SELECT ENAME,SAL*12 AS "ANNUAL SAL" FROM EMP ;

=> display ENAME EXPERIENCE ?

SELECT ENAME,
ROUND((SYSDATE-HIREDATE)/365) AS EXPERIENCE
FROM EMP ;

=> display ENAME SAL COMM TOTSAL ?

TOTSAL = SAL + COMM

```
SELECT ENAME,SAL,COMM,SAL+COMM AS TOTSAL FROM EMP ;
```

SMITH	800	NULL	NULL
ALLEN	1600	300	1900

```
SELECT ENAME,SAL,COMM,SAL+ NVL(COMM,0) TOTSAL FROM EMP ;
```

SMITH	800	NULL	800
ALLEN	1600	300	1900

=> display ENAME SAL HRA DA TAX TOTSAL ?

HRA = house rent allowance = 20% on sal

DA = dearness allowance = 30% on sal

TAX = 10% on sal

TOTSAL = SAL + HRA + DA - TAX

```
SELECT ENAME,SAL,
       SAL*0.2 AS HRA,
       SAL*0.3 AS DA,
       SAL*0.1 AS TAX,
       SAL + (SAL*0.2) + (SAL*0.3) - (SAL*0.1) AS TOTSAL
FROM EMP ;
```

SMITH	800	160	240	80	1120
-------	-----	-----	-----	----	------

=> display SNO TOTAL AVG ?

STUDENT				
SNO	SNAME	S1	S2	S3
1	A	80	90	70
2	B	30	60	50

```
SELECT SID, S1+S2+S3 AS TOTAL , (S1+S2+S3)/3 AS AVG FROM STUDENT ;
```

1	240	80
2	140	46.

ORDER BY clause :-

=> order by clause is used to sort table data based on one or more columns either in ascending or in descending order.

```
SELECT columns
FROM tablename
[WHERE cond]
ORDER BY colname ASC/DESC , ----- ;
```

=> arrange employees data name wise asc order ?

```
SELECT EMPNO,ENAME,SAL
FROM EMP
ORDER BY ENAME ASC ;
```

=> arrange employee list sal wise desc order ?

```
SELECT EMPNO,ENAME,SAL
FROM EMP
ORDER BY SAL DESC ;
```

=> arrange employee list dept wise asc and with in dept sal wise desc ?

```
SELECT EMPNO,ENAME,SAL,DEPTNO
FROM EMP
ORDER BY DEPTNO ASC , SAL DESC ;
```

1 A	3000	20		5	E	6000	10
2	B	5000	10		2	B	5000 10
3 C	4000	30	=====>	4	D	5000	20
4 D	5000	20		1	A	3000	20
5 E	6000	10		3	C	4000	30

=> arrange employee list dept wise asc and with in dept hiredate wise asc ?

```
SELECT EMPNO,ENAME,HIREDATE,DEPTNO
FROM EMP
ORDER BY DEPTNO ASC , HIREDATE ASC ;
```

29-apr-24

=> arrange student list avg wise desc ,m desc,p desc ?

STUDENT

SNO	SNAME	M	P	C
1	A	80	90	70
2	B	60	70	50
3	C	90	80	70
4	D	90	70	80

```
SELECT SNO,SNAME,M,P,C,(M+P+C)/3 AS AVG
FROM STUDENT
ORDER BY (M+P+C)/3 DESC , M DESC , P DESC ;
```

3	C	90	80	70	80
4	D	90	70	80	80
1	A	80	90	70	80
2	B	60	70	50	60

=> display employee list working as clerk,manager and arrange output sal wise desc ?

```
SELECT EMPNO,ENAME,JOB,SAL
FROM EMP
WHERE JOB IN ('CLERK','MANAGER')
ORDER BY SAL DESC ;
```

Question :-

=> display employees joined in 1981 year and arrange output name wise asc ?

DISTINCT clause :-

=> DISTINCT clause eliminates duplicates from select stmt output

syn :-

```
SELECT DISTINCT COLNAME
SELECT DISTINCT COL1,COL2,---
SELECT DISTINCT *
```

Ex :-

SQL>SELECT DISTINCT JOB FROM EMP ;

CLERK
SALESMAN
ANALYST
MANAGER
PRESIDENT

SQL>SELECT DISTINCT DEPTNO FROM EMP ;

10
20
30

SQL> SELECT DISTINCT DEPTNO,JOB FROM EMP ;

DEPTNO	JOB
20	MANAGER
20	ANALYST
10	PRESIDENT
10	CLERK
30	SALESMAN
10	MANAGER
20	CLERK
30	MANAGER
30	CLERK

FETCH clause :-

=> used to display first <n> rows from table

syn :-

SELECT columns
FROM tabname
[WHERE cond]
[ORDER BY col ASC/DESC]
FETCH FIRST <N> ROWS ONLY ;

=> display first 5 rows from emp table ?

```
SELECT EMPNO,ENAME,SAL
FROM EMP
FETCH FIRST 5 ROWS ONLY ;
```

```
SELECT EMPNO,ENAME,SAL
FROM EMP
FETCH FIRST 50 PERCENT ROWS ONLY ;
```

=> display 5th row ?

```
SELECT EMPNO,ENAME,SAL
FROM EMP
OFFSET 4 ROWS FETCH NEXT 1 ROW ONLY ;
```

=> display 5th to 10th record ?

```
SELECT EMPNO,ENAME,SAL
FROM EMP
OFFSET 4 ROWS FETCH NEXT 6 ROWS ONLY ;
```

=> display top 5 highest paid employees ?

```
SELECT EMPNO,ENAME,SAL
FROM EMP
ORDER BY SAL DESC
FETCH FIRST 5 ROWS ONLY ;
```

=> display top 3 max salaries ?

```
SELECT DISTINCT SAL
FROM EMP
ORDER BY SAL DESC
FETCH FIRST 3 ROWS ONLY ;
```

=> display 5th max salary ?

```
SELECT DISTINCT SAL
FROM EMP
ORDER BY SAL DESC
OFFSET 4 ROWS FETCH NEXT 1 ROW ONLY ;
```

=> display top 3 employees based on experience ?

```
SELECT EMPNO,ENAME,HIREDATE
FROM EMP
ORDER BY HIREDATE ASC
FETCH FIRST 3 ROWS ONLY ;
```

summary :-

WHERE	=>	to select specific rows from table
ORDER BY	=>	to sort table data
DISTINCT	=>	to eliminate duplicates
FETCH	=>	to select top N rows

30-APR-24

DML commands :- (Data Manipulation Lang)

```
INSERT
UPDATE
DELETE
MERGE
INSERT ALL
```

=> DML commands acts on table data.
=> DML commands acts on instance (ram).
=> to save the operation execute commit.
=> to cancel the operation execute rollback.

UPDATE command :-

=> command used to modify table data.
=> we can update all rows or specific rows
=> we can update single column or multiple columns

syn :-

```
UPDATE <TABNAME>
SET COLNAME = VALUE , COLNAME = VALUE ,-----
[WHERE COND] ;
```

Ex :-

=> update all employees comm with 500 ?

```
UPDATE EMP SET COMM = 500 ;
```

=> update employees comm with 500 whose comm = null ?

```
UPDATE EMP SET COMM = 500 WHERE COMM IS NULL ;
```

=> update employees comm with 800 whose empno = 7844 ;

```
UPDATE EMP SET COMM = 800 WHERE EMPNO = 7844 ;
```

=> update employees comm with null whose comm <> null ?

```
UPDATE EMP SET COMM = NULL WHERE COMM IS NOT NULL ;
```

```
NULL assignment  =  
NULL comparison IS
```

=> increment sal by 20% and comm by 10% those working as salesman and joined in 1981 year ?

```
UPDATE EMP  
SET SAL = SAL + (SAL*0.2) , COMM = COMM + (COMM * 0.1)  
WHERE JOB='SALESMAN'  
AND  
HIREDATE LIKE '%81' ;
```

=> transfer all employees from 10th dept to 20th dept ?

```
UPDATE EMP SET DEPTNO = 20 WHERE DEPTNO = 10 ;
```

Question :-

=> increase all the samsung,redmi,realme mobile phones and tvs price by 10% ?

products

prodid pname price category brand

DELETE command :-

=> command used to delete row/rows from table

=> we can delete all rows or specific rows

syn :- DELETE FROM <tablename> [WHERE cond] ;

ex :-

=> delete all employees having more than 40 years of expr ?

```
SQL>DELETE FROM EMP WHERE (SYSDATE-HIREDATE)/365 > 40 ;
```

=> delete employee joined in last 5 years ?

```
DELETE FROM EMP WHERE HIREDATE > SYSDATE - (5*365) ;
```

```
DELETE FROM EMP WHERE (SYSDATE-HIREDATE)/365 <= 5 ;
```

=> delete all rows ?

```
DELETE FROM EMP ;
```

FLASHBACKING DATA :-

=> using flashback we can see the data that exists some time back

=> a normal query returns current data but a query that returns past data is called flashback query

=> useful to recover data after commit.

syn :-

```
SELECT columns  
FROM tablename  
AS OF TIMESTAMP ( SYSDATE - INTERVAL );
```

Ex :-

=> the following query returns 5 mins back data in emp table ?

```
SELECT EMPNO,ENAME,SAL,COMM  
FROM EMP  
AS OF TIMESTAMP (SYSDATE - INTERVAL '5' MINUTE) ;
```

NOTE :- we can flashback upto 900secs (15mins)

recovering data after commit :-

STEP 1 :-

SQL>DELETE FROM EMP ;

STEP 2 :-

SQL> COMMIT ;

STEP 3 :-

=> to recover data after commit , get the data that exists 5 mins back and insert that data into current emp table.

```
INSERT INTO EMP
SELECT *
FROM EMP
AS OF TIMESTAMP(SYSDATE - INTERVAL '5' MINUTE) ;
```

DDL commands :- (Data Definition Lang)

create
alter
drop
truncate
rename
flashback
purge

=> all DDL commands acts on table structure.

=> all DDL commands are auto committed.

DDL command = DDL command + COMMIT

Example 1 :-

create table a(a number(2)); => COMMIT

```
insert into a values(10);
insert into a values(20);
insert into a values(30);
insert into a values(40);
rollback ;
```

```
create table => saved
inserts      => cancelled
```

Example 2 :-

```
create table a(a number(2));  => COMMIT
insert into a values(10);
insert into a values(20);
create table b(b number(2));  => COMMIT
insert into a values(30);
insert into a values(40);
rollback ;
```

```
create table a => saved
insert 10,20   => saved
create table b => saved
insert 30,40   => cancelled
```

ALTER command :-

=> command used to modify table structure
=> modifying structure means

- 1 adding columns
- 2 dropping columns
- 3 renaming a column
- 4 modifying a column
 - changing datatype
 - change size

Adding new columns :-

syn :-

ALTER TABLE <TABNAME>

```
ADD (COLNAME DATATYPE(SIZE) ,-----);
```

Ex :-

=> add columns gender,dob to emp table ?

```
ALTER TABLE EMP  
ADD (GENDER CHAR(1) , DOB DATE);
```

after adding by default the new column is filled with NULLs , to insert data into the new column

use update command.

```
UPDATE EMP SET GENDER = 'M' WHERE EMPNO = 7369 ;
```

```
UPDATE EMP SET GENDER = 'F' , DOB = '10-FEB-1960' WHERE EMPNO = 7499 ;
```

Dropping columns :-

syn :-

```
ALTER TABLE <TABNAME>  
DROP (COL1,COL2,----) ;
```

ex :-

=> drop columns gender,dob from emp table ?

```
ALTER TABLE EMP  
DROP (GENDER,DOB) ;
```

Renaming a column :-

syn :-

```
ALTER TABLE <TABNAME>  
RENAME COLUMN <OLDNAME> TO <NEWNAME> ;
```

Ex :-

=> rename column comm to bonus ?

```
ALTER TABLE EMP  
  RENAME COLUMN COMM TO BONUS ;
```

Question :-

- 1 SELECT EMPNO , ENAME,SAL,COMM AS BONUS FROM EMP ;
- 2 ALTER TABLE EMP
 RENAME COLUMN COMM TO BONUS ;

difference between alias and rename ?

ans :-

=> alias changes column heading in select stmt output

=> rename changes column name in table.

Modifying a column :-

1 incr / decr field size

2 changing datatype

```
ALTER TABLE <TABNAME>  
  MODIFY (COLNAME DATATYPE(SIZE) );
```

Ex :-

=> increase the size of ename to 20 ?

```
ALTER TABLE EMP  
  MODIFY (ENAME VARCHAR2(20));
```

```
ALTER TABLE EMP  
  MODIFY (ENAME VARCHAR2(5)); => ERROR => some names contains more than 5  
chars
```

NOTE :-

=> numeric field must be empty to decrease precision or scale

```
ALTER TABLE EMP
  MODIFY (SAL NUMBER(6,2)); => ERROR
```

=> field must be empty to change datatype

```
ALTER TABLE EMP
  MODIFY (EMPNO VARCHAR2(10)); => ERROR
```

how to change datatype of a column even if it contains data ?

modify empno column datatype to varchar2 ?

STEP 1 :- add column empid to emp table

```
ALTER TABLE EMP
  ADD (EMPID VARCHAR2(10));
```

STEP 2 :- copy values from empno to empid

```
UPDATE EMP SET EMPID = 'TCS'||EMPNO ;
```

EMPNO	EMPID
7369	TCS7369

STEP 3 :- remove column empno from emp table

```
ALTER TABLE EMP
  DROP (EMPNO) ;
```

STEP 4 :- rename column empid to empno

```
ALTER TABLE EMP
  RENAME COLUMN EMPID TO EMPNO ;
```

02-may-24

DROP command :-

=> drops table from db

=> drops table structure along with data

syn :- DROP TABLE <TABNAME>

ex :-

SQL>DROP TABLE EMP ;

=> before 10g ver when table is dropped then it is permanently removed and cannot be restored but from 10g onwards when table is dropped then it is moved to recyclebin

=> to see the recyclebin

SQL>SHOW RECYCLEBIN

FLASHBACK command :-

=> command used to restore table from recyclebin

=> useful to recover tables when accidentally dropped

syn :- FLASHBACK TABLE <TABNAME> TO BEFORE DROP ;

Ex :-

SQL>FLASHBACK TABLE EMP TO BEFORE DROP ;

=> table is restored with columns and rows that exists before drop

PURGE command :-

=> command used to delete table from recyclebin

syn :- PURGE TABLE <TABNAME> ;

ex :-

SQL>PURGE TABLE EMP ;

=> after deleting table from recyclebin we cannot flashback the table.

DROP & PURGE :-

SQL>DROP TABLE STUDENT PURGE ;

=> table is dropped and also deleted from recyclebin

TRUNCATE command :-

=> deletes all the data from table but keeps structure

=> will empty the table

=> releases memory allocated for table

SYN :- TRUNCATE TABLE <TABNAME> ;

ex :-

SQL>TRUNCATE TABLE EMP ;

=> when above command is executed oracle goes to memory and releases all the blocks allocated for table and when blocks are released data stored in blocks also deleted.

DROP VS DELETE VS TRUNCATE :-

	DROP	DELETE/TRUNCATE
	drops structure with data	deletes only data but not structure
	DELETE	TRUNCATE
1	DML	DDL
2	we can all rows and specific rows	we can delete only all rows but cannot delete specific row
3	where cond can be used with delete	where cond cannot be used with truncate
4	operations can be rolledback	cannot be rolledback

5	we can flashback after delete	we cannot flashback after truncate
6	deletes row-by-row	deletes all rows at a time
7	slower	faster
8	delete will not release memory	releases memory

RENAME :-

=> command used to change tablename

syn :- RENAME <oldname> TO <newname>

ex :-

=> rename table emp to employees ?

SQL>RENAME EMP TO EMPLOYEES ;

Built-in functions in oracle :-

=> a function accepts some input performs some calculation and returns one value

Types of functions :-

1 single row functions

- character
- numeric
- date
- conversion
- special
- analytical

2 multi row functions (group functions)

single row functions :-

=> these functions process one row at a time and returns one value per row

character functions :-

UPPER() :-

=> converts string to uppercase

UPPER(arg)

ex :-

SQL>SELECT UPPER('hello') FROM DUAL ; => HELLO

LOWER() :-

=> converts string to lowercase

SQL>SELECT LOWER('HELLO') FROM DUAL ; => hello

=> display EMPNO ENAME SAL ? display names in lowercase ?

SELECT EMPNO,LOWER(ENAME) AS ENAME,SAL FROM EMP ;

=> convert names to lowercase in table ?

UPDATE EMP SET ENAME = LOWER(ENAME) ;

03-may-24

LENGTH() :-

SQL>SELECT ENAME,LENGTH(ENAME) FROM EMP ;

=> display employees name contains 4 chars ?

SELECT *
FROM EMP
WHERE LENGTH(ENAME) = 4 ;

SUBSTR() :-

=> returns part of the string

SUBSTR(string,start,[no of chars])

ex :-

SUBSTR('HELLO WELCOME',1,5) => HELLO

SUBSTR('HELLO WELCOME',10,3)=> COM

SUBSTR('HELLO WELCOME',7) => WELCOME

SUBSTR('HELLO WELCOME', -5,3) => LCO

SUBSTR('HELLO WELCOME',-7) => WELCOME

=> employees name starts with 's' ?

SELECT * FROM EMP WHERE SUBSTR(ENAME,1,1) = 's' ;

=> employees name ends with 's' ?

SELECT * FROM EMP WHERE SUBSTR(ENAME,-1,1) = 's' ;

=> employees name starsts and ends with same char ?

SELECT * FROM EMP WHERE ENAME LIKE 'a%a'
OR
ENAME LIKE 'b%b'

SELECT * FROM EMP WHERE SUBSTR(ENAME,1,1) = SUBSTR(ENAME,-1,1) ;

=> generate emailids for employees ?

empno	ename	emailid
7369	smith	smi736@tcs.com
7499	allen	all749@tcs.com

SELECT EMPNO,ENAME,
SUBSTR(ENAME,1,3)||SUBSTR(EMPNO,1,3)||'@tcs.com' AS EMAILID
FROM EMP ;

=> store emailids in db ?

STEP 1 :- add emailid column to emp table

```
ALTER TABLE EMP
ADD (EMAILID VARCHAR2(20));
```

STEP 2 :- update the column with emailids

```
UPDATE EMP
SET EMAILID = SUBSTR(ENAME,1,3)||SUBSTR(EMPNO,1,3)||'@tcs.com' ;
```

INSTR() :-

=> returns position of a character in a string

INSTR(string,char,[start,occurrence])

ex :- H E L L O W E L C O M E
 1 2 3 4 5 6 7 8 9 10 11 12 13

```
INSTR('HELLO WELCOME','O')   =>    5
INSTR('HELLO WELCOME','X')   =>    0
INSTR('HELLO WELCOME','O',1,2) =>   11
INSTR('HELLO WELCOME','E',1,3)=>`  13
INSTR('HELLO WELCOME','L', -5,2) =>   4
INSTR('HELLO WELCOME','E',-5,2) =>   2
```

=> employees name contains 'A' ?

```
SELECT * FROM EMP WHERE INSTR(ENAME,'a') <> 0 ;
```

=> display CID FNAME LNAME ?

```
CUST
CID CNAME
10  sachin tendulkar
11  virat kohli
```

FNAME = SUBSTR(STRING,START,NO OF CHARS)

SUBSTR(CNAME,1, INSTR(CNAME,' ')-1)

LNAME = SUBSTR(CNAME,INSTR(CNAME,'')+1)

```
SELECT CID,
       SUBSTR(CNAME,1,INSTR(CNAME,' ')-1) AS FNAME,
       SUBSTR(CNAME,INSTR(CNAME,'')+1) AS LNAME
FROM CUST ;
```

=> DISPLAY CID FNAME MNAME LNAME ?

```
CUST
CID CNAME
10 sachin ramesh tendulkar
11 mahendra singh dhoni
```

04-MAY-24

REPLACE() :-

=> used to replace one string with another string

REPLACE(str1,str2,str3)

ex :-

REPLACE('hello','ell','abc')	=>	habco
REPLACE('hello','l','abc')	=>	heabcabco
REPLACE('hello','elo','abc')	=>	hello
REPLACE('@@he@@ll@@o@@@','@','")	=>	hello

TRANSLATE() :-

=> used to translate one char to another char

TRANSLATE(str1,str2,str3)

ex :-

TRANSLATE('hello','elo','abc') => habbc

e => a
l => b
o => c

TRANSLATE('hello','elo',"") => NULL

NOTE :-

=> translate function can be used to encrypt data i.e. converting plain text to cipher text

ex :-

```
SELECT ENAME, TRANSLATE(SAL , '0123456789','$t*q@#%^!') AS SAL FROM EMP ;
```

```
JONES    2975  T!%@
```

=> remove all special chars from '@*HE#\$LL%^O!' ?

```
TRANSLATE( '@*HE#$LL%^O!' , '@*#$%^!' , '*****') => **HE**LL**O*
```

```
SELECT  
REPLACE (TRANSLATE( '@*HE#$LL%^O!' , '@*#$%^!' , '*****') , '*', '')  
FROM DUAL ;
```

NUMERIC FUNCTIONS :-

ROUND
TRUNC
CEIL
FLOOR

```
38.456789 => 38  
           38.45  
           38.4567
```

ROUND() :-

=> used to round number to integer or to decimal places based on avg

```
ROUND(number,[decimal places])
```

ex :-

ROUND(38.5768) => 39

38-----38.5-----39

number < avg => rounded to lowest
number >= avg => rounded to highest

ROUND(38.4768) => 38

ROUND(38.5768,2) => 38.58

ROUND(38.5748,2) => 38.57

ROUND(38.5768,3) => 38.577

ROUND(386,-2) => 400

300-----350-----400

ROUND(386,-1) => 390

380-----385-----390

ROUND(386,-3) => 0

0-----500-----1000

=> DISPLAY ENAME EXPERIENCE round experience to integer ?

```
SELECT ENAME ,  
       ROUND((SYSDATE-HIREDATE)/365) AS EXPR  
FROM EMP ;
```

=> round sal to hundreds in table ?

```
UPDATE EMP SET SAL = ROUND(SAL,-2) ;
```

TRUNC() :-

=> rounds number always to lowest

TRUNC(number,[decimal places])

ex :-

TRUNC(38.8765)	=>	38
TRUNC(38.8765,2)	=>	38.87
TRUNC(386,-2)	=>	300
TRUNC(999,-3)	=>	0

CEIL() :-

=> rounds number always to highest

CEIL(number)

EX :-

CEIL(3.1)	=>	4
-----------	----	---

FLOOR() :-

=> rounds number always to lowest

FLOOR(number)

ex :-

FLOOR(3.9)	=>	3
------------	----	---

Question :-

SELECT ROUND(4567,-1),ROUND(4567,-2),ROUND(4567,-3) FROM DUAL ;

o/p :- 4570 4600 5000

Question :-

```
SELECT ROUND(TRUNC(4567,-1),-3) FROM DUAL ;
```

4560

O/ P:- 5000

DATE functions :-

SYSDATE + 10	=>	10 days added to sysdate
SYSDATE - 10	=>	10 days subtracted from sysdate
SYSDATE-HIREDATE	=>	calculates difference in days
SYSDATE + HIREDATE	=>	INVALID

ADD_MONTHS() :-

=> used to add / subtract months to / from a date

ADD_MONTHS(DATE,MONTHS)

Ex :-

ADD_MONTHS(SYSDATE,1)	=>	04-JUN-24
ADD_MONTHS(SYSDATE,-2)	=>	04-MAR-24

Ex :-

GOLD_RATES	
DATEID	RATE
01-JAN-20	?
02-JAN-20	?

04-MAY-24 ?

=> display today's gold rate ?
=> display yesterday's gold rate ?
=> display last month same day gold rate ?
=> display last year same day gold rate ?

1 SELECT * FROM GOLD_RATES WHERE DATEID = SYSDATE ;

2 SELECT * FROM GOLD_RATES WHERE DATEID = SYSDATE-1 ;

3 SELECT * FROM GOLD_RATES WHERE DATEID = ADD_MONTHS(SYSDATE,-1);

4 SELECT * FROM GOLD_RATES WHERE DATEID = ADD_MONTHS(SYSDATE,-12)

06-MAY-24

MONTHS_BETWEEN() :-

=> returns no of months between two dates

MONTHS_BETWEEN(date1,date2)

ex :-

MONTHS_BETWEEN(SYSDATE,'06-MAY-23') =>

=> display ENAME EXPERIENCE in months ?

```
SELECT ENAME,  
       FLOOR(MONTHS_BETWEEN(SYSDATE,HIREDATE)) AS EXPR  
FROM EMP ;
```

=> display ENAME EXPERIENCE ?
M YEARS N MONTHS

experience = 40 months = 3 YEARS 4 MONTHS

years = months/12 = 40/12 = 3

months = mod(months,12) = mod(40,12) = 4

```
SELECT ENAME,  
       FLOOR(MONTHS_BETWEEN(SYSDATE,HIREDATE)/12) AS YEARS,  
       MOD(FLOOR(MONTHS_BETWEEN(SYSDATE,HIREDATE)),12) AS MONTHS  
FROM EMP ;
```

CONVERSION FUNCTIONS :-

=> used to convert one type to another type

=> oracle provided 3 functions to convert

- 1 TO_CHAR
- 2 TO_DATE
- 3 TO_NUMBER

converting date to char type :-

=> dates converted to char type to display dates in different formats

TO_CHAR(DATE,'format')

formats :- (DATE = SYSDATE)

yyyy	2024
yy	24
mm	05
mon	may
month	may
ddd	127 (day of the year)
dd	06 (day of the month)
d	2 (day of the week)
dy	mon
day	monday
hh	11 (1-12)
hh24	11 (1-24)
mi	56
ss	20
AM/PM	AM time OR PM time
Q	2 (1-4)
	jan-mar 1
	apr-jun 2
	jul-sep 3
	oct-dec 4

```
SELECT TO_CHAR(SYSDATE,'yyyy yy') FROM DUAL ;
```

=> list of employees joined in 1980,1983,1985 ?

```
SELECT *  
FROM EMP  
WHERE TO_CHAR(HIREDATE,'YYYY') IN (1980,1983,1985) ;
```

=> list of employees joined in leap year ?

```
SELECT *  
FROM EMP  
WHERE MOD(TO_CHAR(HIREDATE,'YYYY'),4) = 0 ;
```

=> list of employees joined in jan,apr,dec months ?

```
SELECT *  
FROM EMP  
WHERE TO_CHAR(HIREDATE,'MM') IN (1,4,12) ;
```

=> display smith joined on wednesday
allen joined on friday ?

```
SELECT ENAME||' joined on '||TO_CHAR(HIREDATE,'day')  
FROM EMP ;
```

=> list of employees joined on sunday ?

```
SELECT *  
FROM EMP  
WHERE TO_CHAR(HIREDATE,'d') = 1 ;
```

```
TO_CHAR(HIREDATE,'dy') = 'sun' ;
```

```
TO_CHAR(HIREDATE,'day') = 'sunday' ; => no rows
```

```
TO_CHAR(HIREDATE,'fmday') = 'sunday' ;
```

fm => format mask => removes extra spaces

=> list of employees joined in 2nd quarter of 1981 year ?

```
SELECT *
```

```
FROM EMP
WHERE TO_CHAR(HIREDATE,'YYYY-Q') = '1981-2' ;
```

Ex :-

```
INSERT INTO EMP(EMPNO,ENAME,SAL,HIREDATE) VALUES(100,'A',4000,SYSDATE);
```

=> list of employees joined today ?

```
SELECT *
FROM EMP
WHERE HIREDATE = SYSDATE ;    => NO ROWS
```

06-MAY-24 12:37:20 = 06-MAY-24 12:39:30

```
SELECT *
FROM EMP
WHERE TO_CHAR(HIREDATE,'DD-MM-YYYY') = TO_CHAR(SYSDATE,'DD-MM-YYYY') ;
```

07-may-24

converting number to char type :-

=> numbers converted to char type to display numbers in different formats

TO_CHAR(number,'format')

formats :-

9	represents a digit
0	represents a digit
G	thousand separator
D	decimal separator
L	currency symbol
C	currency

ex :-

TO_CHAR(1234,'999999')	=>	1234
TO_CHAR(1234,'000000')	=>	01234
TO_CHAR(1234,'000000')	=>	001234

TO_CHAR(1234,'9G999')	=>	1,234
TO_CHAR(1234,'9G999D99')	=>	1,234.00
TO_CHAR(1234,'L9G999')	=>	\$1,234
TO_CHAR(1234,'C9G999')	=>	USD1,234

=> display ENAME SAL ? display salaries with thousand separator and currency symbol ?

```
SELECT ENAME,
       TO_CHAR(SAL,'C99G999') AS SAL
FROM EMP ;
```

how to change currency ?

```
ALTER SESSION SET NLS_TERRITORY='INDIA' ;
```

converting date string to date :-

```
DATE STRING => '07-MAY-24'
              '05/07/24'
              '2024-05-07'
```

```
TO_DATE(DATE STRING,'FORMAT')
```

Ex :-

```
SQL> SELECT SYSDATE + 10 FROM DUAL ; => 17-MAY-24
```

```
SQL>SELECT '01-JAN-24' + 200 FROM DUAL ; => ERROR
```

```
number + number => valid
date + number   => valid
string + number => invalid
```

```
SQL>SELECT TO_DATE('01-JAN-24','DD-MON-YY') + 200 FROM DUAL ; => 19-JUL-24
```

calculate '05/07/24' + 30 ?

```
SQL>SELECT TO_DATE('05/07/24','MM/DD/YY') + 30 FROM DUAL ;
```

=> write a query to display on which day india got independence ?

```
SELECT TO_CHAR(TO_DATE('&DATE','DD-MON-YYYY'),'DAY')
```

FROM DUAL ;

How to implement loop in sql :-

SELECT LEVEL
FROM DUAL
CONNECT BY LEVEL <= 10;

```
LEVEL = 1
    WHILE(LEVEL<=10)
    {
        PRINT LEVEL
        LEVEL = LEVEL + 1
    }
```

=> LEVEL is a system variable.

=> LEVEL initialized to 1

=> by default LEVEL is incremented by 1

=> print even nos upto 20 ?

SELECT LEVEL
FROM DUAL
WHERE MOD(LEVEL,2) = 0
CONNECT BY LEVEL <= 20;

=> write a query to print 2024 calendar ?

01-JAN-24 ?

02-JAN-24 ?

31-DEC-24 ?

SELECT LEVEL + TO_DATE('31-DEC-2023','DD-MON-YYYY') AS DATEID ,
 TO_CHAR(LEVEL + TO_DATE('31-DEC-2023','DD-MON-YYYY') , 'DAY') AS DAY
FROM DUAL
CONNECT BY LEVEL <= 366 ;

INPUT	OUTPUT	FUNCTION
-------	--------	----------

DD-MON-YY	MM/DD/YY	TO_CHAR
MM/DD/YY	DD-MON-YY	TO_DATE

ORACLE	OTHER	TO_CHAR
OTHER	ORACLE	TO_DATE

5000	\$5,000	TO_CHAR
\$5,000	5000	TO_NUMBER

special functions :-

NVL() :-

=> used to convert null values

NVL(arg1,arg2)

ex :-

if arg1 = null returns arg2

if arg1 <> null returns arg1 only

ex :-

NVL(100,200) => 100

NVL(NULL,200) => 200

08-MAY-24

=> display ENAME SAL COMM TOTSAL ?

SELECT ENAME,SAL,COMM,SAL+COMM AS TOTSAL FROM EMP ;

smith	800	null	null
allen	1600	300	1900

SELECT ENAME,SAL,COMM,SAL+NVL(COMM,0) AS TOTSAL FROM EMP ;

smith	800	null	800
allen	1600	300	1900

Analytical functions :-

RANK & DENSE_RANK :-

- => both functions are used to find ranks
- => ranking is always based on some column
- => for rank functions data must be sorted

```
RANK() OVER (ORDER BY COL ASC/DESC)
DENSE_RANK() OVER (ORDER BY COL ASC/DESC)
```

Ex :-

- => find ranks of the employees based on sal and highest paid should get 1st rank ?

```
SELECT ENAME,SAL,
       RANK() OVER (ORDER BY SAL DESC) AS RNK
FROM EMP ;
```

```
SELECT ENAME,SAL,
       DENSE_RANK() OVER (ORDER BY SAL DESC) AS RNK
FROM EMP ;
```

difference between rank & dense_rank ?

- 1 rank function generates gaps but dense_rank will not generate gaps
- 2 in rank function ranks may not be in sequence but in dense_rank ranks are always in sequence

SAL	RNK	DRNK
5000	1	1
4000	2	2
3000	3	3
3000	3	3
3000	3	3
2000	6	4
2000	6	4
1000	8	5

- => find the ranks based on sal desc , if salaries are same then ranking should be based on hiredate ?

```
SELECT ENAME,HIREDATE,SAL,
       DENSE_RANK() OVER (ORDER BY SAL DESC,HIREDATE ASC) AS RNK
FROM EMP ;
```

GROUP functions :-

=> these functions process group of rows and returns one value

=> these functions are also called multi row functions or aggregate functions

MAX
MIN
SUM
AVG
COUNT
COUNT(*)

MAX() :-

=> returns maximum value

MAX(arg)

Ex :-

SELECT MAX(SAL) FROM EMP ;	=>	5000
SELECT MAX(HIREDATE) FROM EMP ;	=>	12-JAN-83
SELECT MAX(ENAME) FROM EMP ;	=>	ward

MIN() :-

=> returns minimum value

MIN(arg)

SELECT MIN(SAL) FROM EMP ;	=>	800
SELECT MIN(HIREDATE) FROM EMP ;	=>	17-DEC-80
SELECT MIN(ENAME) FROM EMP ;	=>	ADAMS

SUM() :-

=> returns total

SUM(arg)

SELECT SUM(SAL) FROM EMP ; => 33025

=> round total sal to hundreds ?

SELECT ROUND(SUM(SAL),-2) FROM EMP ; => 33000

33000-----33050-----33100

=> after rounding display total sal with thousand separator and currency ?

```
SELECT
  TO_CHAR( ROUND(SUM(SAL),-2) , 'C99G999')
FROM EMP ;
```

o/p :- USD33,000

=> calculate total sal including comm ?

SELECT SUM(SAL+COMM) FROM EMP ; => 7800

SELECT SUM(SAL+NVL(COMM,0)) FROM EMP ; => 35225

AVG() :-

=> returns avg value

AVG(arg)

ex :-

SELECT AVG(SAL) FROM EMP ; => 2201.66667

=> round avg sal to lowest integer ?

SELECT FLOOR(AVG(SAL)) FROM EMP ;

2201-----2202

note :- SUM,AVG cannot be applied on char,date columns

=> avg sal paid to managers ?

SELECT AVG(SAL) FROM EMP WHERE JOB='MANAGER' ;

COUNT() :-

=> returns no of values in a column

COUNT(arg)

SELECT COUNT(EMPNO) FROM EMP ; => 15

SELECT COUNT(COMM) FROM EMP ; => 4 => nulls not counted

COUNT(*) :-

=> returns no of rows in a table

SELECT COUNT(*) FROM EMP ; => 15

T1

F1

10

NULL

20

NULL

30

COUNT(F1) => 3 => nulls are not counted

COUNT(*) => 5 => nulls are counted

=> how many employeeed joined in 1981 year ?

SELECT COUNT(*) FROM EMP WHERE TO_CHAR(HIREDATE,'YYYY') = 1981 ;

=> how many employees joined on sunday ?

SELECT COUNT(*) FROM EMP WHERE TO_CHAR(HIREDATE,'DY')='SUN' ;

=> how many employees joined in 2nd quarter of 1981 year ?

SELECT COUNT(*)

```
FROM EMP
WHERE TO_CHAR(HIREDATE,'YYYY-Q') = '1981-2' ;
```

NOTE :-

=> group functions are not allowed in where clause and they are allowed only in select , having clauses.

```
SELECT ENAME
FROM EMP
WHERE SAL = MAX(SAL) ; => ERROR
```

```
SELECT DEPTNO
FROM EMP
WHERE COUNT(*) = 3 ; => ERROR
```

STRING	:-	upper,lower,length,substr,instr,replace,translate
NUMERIC	:-	round,trunc,ceil,floor,mod
DATE	:-	add_months,months_between
CONVERSION	:-	to_char,to_date,to_number
SPECIAL	:-	nvl
ANALYTICAL	:-	rank,dense_rank
GROUP	:-	max,min,sum,avg,count,count(*)

How to implement IF-ELSE :-

=> case statement is used to implement IF-ELSE
=> using case statement we can return values based on conditions
=> case statements are 2 types

- 1 simple case
- 2 searched case

simple case :-

=> use simple case when conditions based on "=" operator

```
CASE COLNAME
WHEN VALUE1 THEN RETURN EXPR1
WHEN VALUE2 THEN RETURN EXPR2
WHEN VALUE3 THEN RETURN EXPR3
```

ELSE RETURN EXPR
END

=> display ENAME DNAME ?

IF DEPTNO =10	DISPLAY	ACCOUNTS
20		RESEARCH
30		SALES
OTHERS		UNKNOWN

```
SELECT ENAME,  
       CASE DEPTNO  
         WHEN 10 THEN 'ACCOUNTS'  
         WHEN 20 THEN 'RESEARCH'  
         WHEN 30 THEN 'SALES'  
         ELSE ' UNKNOWN'  
       END AS DNAME  
FROM EMP ;
```

09-may-24

=> increment employee salaries as follows ?

if job=CLERK	incr sal by	10%
SALESMAN		15%
MANAGER		20%
others		5%

```
UPDATE EMP  
SET SAL = CASE JOB  
             WHEN 'CLERK' THEN SAL + (SAL*0.1)  
             WHEN 'SALESMAN' THEN SAL + (SAL*0.15)  
             WHEN 'MANAGER' THEN SAL + (SAL*0.2)  
             ELSE SAL+(SAL*0.05)  
           END ;
```

searched case :-

=> use searched case when conditions not based on "=" operator.

CASE

```

WHEN COND1 THEN RETURN EXPR1
WHEN COND2 THEN RETURN EXPR2
WHEN COND3 THEN RETURN EXPR3

```

```

-----
ELSE RETURN EXPR
END

```

=> display ENAME SAL SALRANGE ?

```

0-2000
2001-4000
ABOVE 4000

```

```

SELECT ENAME,SAL,
       CASE
         WHEN SAL BETWEEN 0 AND 2000 THEN '0-2000'
         WHEN SAL BETWEEN 2001 AND 4000 THEN '2001-4000'
         WHEN SAL>4000 THEN 'ABOVE 4000'
       END AS SALRANGE
FROM EMP ;

```

=> display SNO TOTAL AVG RESULT ?

STUDENT				
SNO	SNAME	S1	S2	S3
1	A	80	90	70
2	B	30	60	50

```

SELECT SNO,
       S1+S2+S3 AS TOTAL,
       ROUND((S1+S2+S3)/3 ,2) AS AVG,
       CASE
         WHEN S1>=35 AND S2>=35 AND S3>=35 THEN 'PASS'
         ELSE 'FAIL'
       END AS RESULT
FROM STUDENT ;

```

```

=====
=

```

```

INTEGRITY CONSTRAINTS
=====

```

=> Integrity constraints are rules to maintain data integrity i.e. data quality or data consistency.

=> prevent users from entering invalid data.

=> used to enforce rules like min bal must be 1000

Types of constraints :-

- NOT NULL
- UNIQUE
- PRIMARY KEY
- CHECK
- FOREIGN KEY
- DEFAULT

=> above constraints can be declared in two ways

- 1 column level
- 2 table level

column level :-

=> if constraints are declared immediately after declaring column then it is called column level

```
CREATE TABLE <TABNAME>
(
    COLUMNAME DATATYPE(SIZE) CONSTRAINT ,
    ----- ,
    -----
);
```

NOT NULL :-

=> NOT NULL constraint doesn't accept null values

=> a field declared with NOT NULL is called mandatory field

ex :-

```
CREATE TABLE EMP11
(
  EMPNO INT ,
  ENAME VARCHAR(10) NOT NULL
);
```

```
INSERT INTO EMP11 VALUES(1,"); => ERROR
INSERT INTO EMP11 VALUES(1,'A');
```

UNIQUE :-

=> unique constraint doesn't accept duplicates

ex :-

```
CREATE TABLE CUST
(
  CID NUMBER(2),
  CNAME VARCHAR2(10) NOT NULL,
  EMAILID VARCHAR2(20) UNIQUE
);
```

```
INSERT INTO CUST VALUES(10,'A','abc@gmail.com');
INSERT INTO CUST VALUES(11,'B','abc@gmail.com'); => ERROR
INSERT INTO CUST VALUES(12,'C','');
```

NOTE :- unique constraint doesn't accept duplicates but allows nulls.

PRIMARY KEY :-

=> primary key doesn't accept duplicates and nulls

=> it is the combination of unique & not null

=> In tables one column must be there to uniquely identify and into that column duplicates and nulls are not allowed , so declare that column with primary key.

ex :-

```
CREATE TABLE EMP12
(
    EMPNO NUMBER(4) PRIMARY KEY,
    ENAME VARCHAR(10) NOT NULL
);
```

```
INSERT INTO EMP12 VALUES(100,'A');
INSERT INTO EMP12 VALUES(100,'B'); => ERROR
INSERT INTO EMP12 VALUES(NULL,'C'); => ERROR
```

NOTE :- only one primary key is allowed per table. if we want multiple primary keys then declare one column with primary key and other columns with unique not null.

ex :-

```
CREATE TABLE CUST
(
    CUSTID NUMBER(6) PRIMARY KEY,
    CNAME VARCHAR2(10) NOT NULL,
    AADHARNO NUMBER(12) UNIQUE NOT NULL,
    PANNO CHAR(10) UNIQUE NOT NULL
);
```

=> columns declared with UNIQUE NOT NULL are called secondary keys or alternate keys

CHECK constraint :-

=> use check constraint when rule based on condition

CHECK(condition)

example 1 :- sal must be min 3000

```
CREATE TABLE EMP14
(
    EMPNO NUMBER(4) PRIMARY KEY,
    ENAME VARCHAR2(10) NOT NULL,
    SAL NUMBER(7) CHECK(SAL>=3000)
);
```

```
INSERT INTO EMP14 VALUES(100,'A',1000); => ERROR
INSERT INTO EMP14 VALUES(101,'A',5000);
```

INSERT INTO EMP14 VALUES(102,'B',NULL); => ACCEPTED

example 2 :- GENDER MUST BE 'M','F' ?

GENDER CHAR(1) CHECK(GENDER IN ('M','F'))

M

F

D => NOT ALLOWED

example 3 :- AMT must be multiple of 100 ?

AMT NUMBER(7) CHECK(MOD(AMT,100)=0)

5000

2555 => NOT ALLOWED

example 4 :- PWD must be min 6 chars

PWD VARCHAR2(10) CHECK(LENGTH(PWD) >= 6)

example 5 :- emailid must contain '@'

emailid must end with '.com' or '.co' or '.in'

```
EMAILID VARCHAR2(20) CHECK(EMAILID LIKE '%@%'
AND
(
EMAILID LIKE '%.com'
OR
EMAILID LIKE '%.co'
OR
EMAILID LIKE '%.in'
))
```

FOREIGN KEY :-

=> use foreign key to establish relationship between two tables

EMP

EMPNO	ENAME	SAL	DNO	DNAME	LOC
1	A	5000	10	HR	HYD
2	B	4000	10	HR	HYD

3	C	3000	10	HR	HYD
---	---	------	----	----	-----

20			10	HR	HYD
----	--	--	----	----	-----

=> above table contains redundancy i.e. repetition of data and it leads to memory wastage and insert,update,delete anomalies , to to reduce redundancy divide the table into two tables and establish relationship by using foreign key

=> to establish relationship take primary key of one table and add it to another table as foreign key and declare with references constraint.

ex :-

DEPT		
DNO	DNAME	LOC
10	HR	HYD
20	IT	BLR

EMP				
EMPNO	ENAME	SAL	DNO	REFERENCES DEPT(DNO)
1	A	5000	10	
2	B	6000	20	
3	C	7000	90	=> INVALID
4	D	3000	10	
5	E	2000	NULL	

=> values entered in foreign key column should match with values entered in primary key column

=> foreign key allows duplicates and nulls.

=> after declaring foreign key a relationship is created between two tables called parent / child relationship.

=> primary key table is parent and foreign key table is child.

```
CREATE TABLE DEPT55
(
  DNO NUMBER(2) PRIMARY KEY,
  DNAME VARCHAR2(10) UNIQUE NOT NULL
);
```

```
INSERT INTO DEPT55 VALUES(10,'HR');
INSERT INTO DEPT55 VALUES(20,'IT');
```

```
CREATE TABLE EMP55
(
  EMPNO  NUMBER(4) PRIMARY KEY,
  ENAME  VARCHAR2(10) NOT NULL,
  SAL    NUMBER(7) CHECK(SAL BETWEEN 5000 AND 10000) ,
  DNO    NUMBER(2) REFERENCES DEPT55(DNO)
);
```

```
INSERT INTO EMP55 VALUES(1,'A',5000,10);
INSERT INTO EMP55 VALUES(2,'B',6000,90); => ERROR
INSERT INTO EMP55 VALUES(3,'C',7000,NULL);
INSERT INTO EMP55 VALUES(4,'D',8000,10);
```

QUESTION :-

ACCOUNTS

ACCNO	ACTYPE	BAL
-------	--------	-----

Rules :-

- 1 accno should not be duplicate & null
- 2 actype must be 'S' OR 'C'
- 3 bal must be min 1000

TRANSACTIONS

TRID	TTYPE	DATE	TAMT	ACCNO
------	-------	------	------	-------

Rules :-

- 1 trid should not be duplicate & null
- 2 ttype must be 'W' OR 'D'
- 3 tamt must be multiple of 100
- 4 accno should match with accounts table accno

Relationship Types :-

- 1 one to one (1:1)
- 2 one to many (1:M)

- 3 many to one (M:1)
- 4 many to many (M:N)

=> by default oracle creates one to many relationship between two tables

How to establish one to one relationship :-

=> to establish one to one relationship declare foreign key with unique constraint.

DEPT

DNO DNAME

10 HR

20 IT

MGR

MGRNO MNAME START_DT END_DT DNO REFERENCES

DEPT(DNO) UNQUE

1 A 1/ 20/ 10

2 B 21/ 30 20

11-MAY-24

How to establish many to many relationship :-

=> if relationship is many to many then create 3rd table and add primary keys of both tables as foreign keys

ex :-

STUDENT

SID SNAME

1 A

2 B

COURSE

CID CNAME

10 JAVA

11 ORACLE

REGISTRATIONS

SID CID DOR FEE

1 10 ? 5000

1 11 ? 5000

2 10 ? 5000

=> if relationship is one to one then we can add foreign key to any table
=> if relationship is one to many then we must add foreign key to many side table
=> if relationship is many to many then create 3rd and add primary keys of both tables as foreign keys

DEFAULT :-

=> a column can be declared with default value as follows

ex :- HIREDATE DATE DEFAULT SYSDATE

=> while inserting if we skip hiredate then oracle inserts default value

Ex :-

```
CREATE TABLE EMP16
(
  EMPNO NUMBER(4) PRIMARY KEY ,
  ENAME VARCHAR2(10) NOT NULL,
  SAL    NUMBER(7) CHECK(SAL>=3000),
  HIREDATE DATE DEFAULT SYSDATE
);
```

```
INSERT INTO EMP16(EMPNO,ENAME,SAL) VALUES(100,'A',5000);
INSERT INTO EMP16 VALUES(101,'B',6000,'01-JAN-24');
INSERT INTO EMP16 VALUES(102,'C',4000,"");
```

```
SELECT * FROM EMP16 ;
```

EMPNO	ENAME	SAL	HIREDATE
100	A	5000	11-MAY-24
101	B	6000	01-JAN-24
102	C	4000	

TABLE LEVEL :-

=> if constraints are declared after declaring all columns then it is called table level
=> use table level to declare constraint for multiple or combination of columns

Declaring check constraint at table level :-

PRODUCTS

prodid	pname	price	mfd_dt	exp_dt	
100	AAAA	50	01-MAY-24	01-JAN-24	INVALID

rule :- exp_dt > mfd_dt

CREATE TABLE PRODUCTS

```
(
prodid NUMBER(3) PRIMARY KEY,
pname VARCHAR2(10) NOT NULL,
price NUMBER(5),
mfd_dt DATE ,
exp_dt DATE ,
CHECK(exp_dt > mfd_dt)
);
```

INSERT INTO products VALUES(100,'a',100,sysdate,'01-JAN-24'); => error
INSERT INTO products VALUES(101,'B',50,'01-JAN-24',SYSDATE);

composite primary key :-

=> if primary key is declared for combination of columns then it is called composite primary key.

=> In some tables we may not be able to uniquely identify by using single column and we need combination to uniquely identify and that combination should be declared primary key at table level.

ex :-

STUDENT
SID SNAME

1	A
2	B

COURSE
CID CNAME

10	JAVA
11	ORACLE

REGISTRATIONS

SID CID DOR FEE

```

-----
1      10      ?      5000
1      11      ?      5000
2      10      ?      5000

```

```

CREATE TABLE STUDENT
(
  SID NUMBER(2) PRIMARY KEY,
  SNAME VARCHAR2(10) NOT NULL
);

```

```

INSERT INTO STUDENT VALUES(1,'A');
INSERT INTO STUDENT VALUES(2,'B');

```

```

CREATE TABLE COURSE
(
  CID NUMBER(2) PRIMARY KEY,
  CNAME VARCHAR2(10) NOT NULL
);

```

```

INSERT INTO COURSE VALUES(10,'JAVA');
INSERT INTO COURSE VALUES(11,'ORACLE');

```

```

CREATE TABLE REGISTRATIONS
(
  SID  NUMBER(2) REFERENCES STUDENT(SID) ,
  CID  NUMBER(2) REFERENCES COURSE(CID),
  DOR DATE,
  FEE  NUMBER(5),
      PRIMARY KEY(SID,CID)
);

```

```

INSERT INTO REGISTRATIONS VALUES(1,10,SYSDATE,4000);
INSERT INTO REGISTRATIONS VALUES(1,11,SYSDATE,4000);
INSERT INTO REGISTRATIONS VALUES(2,10,SYSDATE,4000);
INSERT INTO REGISTRATIONS VALUES(1,10,SYSDATE,4000); => ERROR

```

Question :-

```

SALES
DATEID      PRODID  CUSTID  QTY  AMT
10-MAY-24   100    10      1    2000
10-MAY-24   100    11      1    2000

```

10-MAY-24	101	10	1	1000
11-MAY-24	100	10	1	2000

=> identify primary key and write create table script ?

14-may-24

composite foreign key :-

=> if combination of columns declared foreign key then it is called composite foreign key.

=> a composite foreign key refers composite primary key.

ex :-

REGISTRATIONS			
SID	CID	DOR	FEE

1	10	?	5000
1	11	?	5000
2	10	?	5000

CERTIFICATES				
CERTNO	DOI		SID	CID
1000	14-	1	10	
1001	14-	1	11	
1002	14-	2	11	=> INVALID

=> in the above example SID,CID combination should match with registrations table sid,cid combination , so declare this combination as foreign key

```

CREATE TABLE CERTIFICATES
(
  CERTNO NUMBER(4) PRIMARY KEY,
  DOI    DATE,
  SID    NUMBER(2),
  CID    NUMBER(2),
        FOREIGN KEY(SID,CID) REFERENCES REGISTRATIONS(SID,CID)
);

```

1 which of the following constraint cannot be declared at table level ?

- A CHECK
- B UNIQUE
- C NOT NULL
- D PRIMARY KEY
- E FOREIGN KEY

ANS :- C

2 Which statements are true regarding constraints ?

- A a foreign key cannot contain NULL value
- B a column with UNIQUE constraint can contain NULL value
- C a constraint is enforced only for the INSERT operation on a table
- D all constraints can be defined at column level and table level.

3 Which CREATE TABLE statement is valid?

- A. CREATE TABLE ord_details
(ord_no NUMBER(2) PRIMARY KEY,
item_no NUMBER(3) PRIMARY KEY,
ord_date DATE NOT NULL);
- B. CREATE TABLE ord_details
(ord_no NUMBER(2) UNIQUE, NOT NULL,
item_no NUMBER(3),
ord_date DATE DEFAULT SYSDATE NOT NULL);
- C. CREATE TABLE ord_details
(ord_no NUMBER(2) ,
item_no NUMBER(3),
ord_date DATE DEFAULT NOT NULL,
UNIQUE (ord_no),
PRIMARY KEY (ord_no));
- D. CREATE TABLE ord_details
(ord_no NUMBER(2),
item_no NUMBER(3),
ord_date DATE DEFAULT SYSDATE NOT NULL,
PRIMARY KEY (ord_no, item_no));

Data Dictionary / Metadata :-

=> oracle not only stores data and it also stores metadata and this metadata organized in tables called system tables or data dictionary tables

ALL_USERS :-

=> stores information about users

list of users in oracle db ?

SQL>SELECT USERNAME FROM ALL_USERS ;

usernames starts with B ?

SQL>SELECT USERNAME FROM ALL_USERS WHERE USERNAME LIKE 'B%' ;

USER_TABLES :-

=> stores list of tables created by user

list of tables created by user ?

SQL>SELECT TABLE_NAME FROM USER_TABLES ;

USER_CONSTRAINTS :-

=> stores information about constraints declared in a table.

list of constraints declared in emp55 table ?

```
SELECT CONSTRAINT_NAME,CONSTRAINT_TYPE,
       SEARCH_CONDITION
FROM USER_CONSTRAINTS
WHERE TABLE_NAME='EMP55' ;
```

CONSTRAINT_NAME	C	SEARCH_CONDITION
-----	-	-----
SYS_C009254	C	"ENAME" IS NOT NULL
SYS_C009255	C	SAL BETWEEN 5000 AND 10000

SYS_C009257	R
SYS_C009256	P

Dropping constraints :-

```
ALTER TABLE <TABNAME>
  DROP CONSTRAINT <NAME>;
```

Ex :-

=> drop check constraint in emp55 table ?

```
ALTER TABLE EMP55
  DROP CONSTRAINT SYS_C009255 ;
```

=> drop primary key in emp55 table ?

```
ALTER TABLE EMP55
  DROP PRIMARY KEY ;
```

=> drop primary key in dept55 table ?

```
ALTER TABLE DEPT55
  DROP PRIMARY KEY ; => ERROR => can't drop pk if referenced by fk
```

DROP TABLE DEPT55 ; => ERROR => can't drop table because referenced by fk

CASCADE :-

```
ALTER TABLE DEPT55
  DROP PRIMARY KEY CASCADE ; => drops pk with dependent fk
```

DROP TABLE DEPT55 CASCADE CONSTRAINTS ; => drops table with dependent fk

summary :-

importance of constraints

types of constraints

declaring constraints

column level

table level

dropping constraints
getting constraints information

=====

==

15-may-24 JOINS

=> join is an operation performed to display data from two or more tables

=> In db related data stored in multiple tables , to gather or to combine data stored in multiple tables we need to join those tables.

ex :-

ORDERS				CUST		
ordid	orddt	deldt	cid	cid	cname	caddr
1000	10	15	10	10	A	HYD
1001	12	17	11	11	B	HYD
1002	13	20	12	12	C	HYD

output :-

ordid	orddt	deldt	cname	caddr
1000	10	15	A	HYD
1001	12	17	B	HYD

Types of joins :-

- 1 INNER JOIN
- 2 OUTER JOIN
 - LEFT OUTER
 - RIGHT OUTER
 - FULL OUTER
- 3 SELF JOIN
- 4 NON EQUI
- 5 CROSS JOIN

INNER JOIN :-

=> to perform inner join between the tables there must be a common field and name of the common field need not to be same and pk-fk relationship is not compulsory.

=> inner join is performed on the common column with same datatype.

syntax :-

```
SELECT columns
FROM tab1 INNER JOIN tab2
  ON join condition ;
```

join condition :-

=> based on the join condition oracle joins the records of two tables.

=> join conditions specifies which record of 1st table joined with which record of 2nd table.

syn :- table1.commonfield = table2.commonfield

Ex :-

EMP				DEPT	
EMPNO	ENAME	SAL	DEPTNO	DEPTNO	DNAME
1	A	3000	10	ACCOUNTS	NEW YORK
2	B	4000	20	RESEARCH	
3	C	6000	30	SALES	
4	D	4000	20	OPERATIONS	
5	E	2000	NULL		

=> display ENAME SAL DNAME LOC ?

EMP DEPT

```
SELECT ENAME,SAL,DNAME,LOC
FROM EMP INNER JOIN DEPT
  ON EMP.DEPTNO = DEPT.DEPTNO ;
```

```
A    ACCOUNTS
B    RESEARCH
```

C SALES
D RESEARCH

note :-

=> In join queries declare table alias and prefix column names with table alias for two reasons

- 1 to avoid ambiguity
- 2 for faster execution

```
SELECT E.ENAME,E.SAL,D.DEPTNO,D.DNAME,D.LOC AS CITY
FROM EMP E INNER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO ;
```

=> display employee details with dept details working at NEW YORK loc ?

```
SELECT E.ENAME,D.DNAME,D.LOC
FROM EMP E INNER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO /* join cond */
WHERE D.LOC = 'NEW YORK' /* filter cond */ ;
```

joining more than 2 tables :-

=> if no of tables increases no of join conditions also increases
=> to join N tables N-1 join condition required

```
SELECT columns
FROM t1 INNER JOIN t2
ON join cond 1
INNER JOIN t3
ON join cond 2
INNER JOIN t4
ON join cond 3
```

example :-

EMP	DEPT	LOCATIONS	COUNTRIES
empno	deptno	locid	country_id
ename	dname	city	country_name

sal	locid	state	
deptno			country_id

=> display ENAME DNAME CITY STATE COUNTRY_NAME ?

-----	-----	-----	-----
EMP	DEPT	LOCATIONS	COUNTRIES

```

SELECT E.ENAME,
       D.DNAME,
       L.CITY,L.STATE,
       C.COUNTRY_NAME
FROM   EMP E INNER JOIN DEPT D
      ON  E.DEPTNO = D.DEPTNO
      INNER JOIN LOCATIONS L
      ON  D.LOCID = L.LOCID
      INNER JOIN COUNTRIES C
      ON  L.COUNTRY_ID = C.COUNTRY_ID ;

```

Question :-

- 1 display order details with cust details ?
- 2 display order details with cust details to be delivered today ?

16-MAY-24

OUTER JOIN :-

=> inner join returns only matching records but it will not return unmatched records ,
to display unmatched records perform outer join

ex :-

EMP				DEPT		
EMPNO	ENAME	SAL	DEPTNO	DEPTNO	DNAME	

LOC

1	A	3000	10	10	ACCOUNTS	NEW YORK
2	B	4000	20	20	RESEARCH	
3	C	6000	30	30	SALES	
4	D	4000	20	40	OPERATIONS	=> unmatched record
5	E	2000	NULL			=> unmatched record

=> outer join is 3 types

- 1 LEFT OUTER JOIN
- 2 RIGHT OUTER JOIN
- 3 FULL OUTER JOIN

LEFT OUTER JOIN :-

=> returns all rows (matched + unmatched) from left side table and matching rows from right side table.

```
SELECT E.ENAME,D.DNAME
FROM EMP E LEFT OUTER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO ;
```

=> returns all rows from emp table and matching rows from dept table

A	ACCOUNTS
B	RESEARCH
C	SALES
D	RESEARCH
E	NULL => unmatched from emp

RIGHT OUTER JOIN :-

=> returns all rows from right side table and matching rows from left side table

```
SELECT E.ENAME,D.DNAME
FROM EMP E RIGHT OUTER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO ;
```

=> returns all rows from dept table and matching rows from emp table

A	ACCOUNTS
B	RESEARCH
C	SALES
D	RESEARCH
NULL	OPERATIONS => unmatched from dept

FULL OUTER JOIN :-

=> returns all rows from both tables

```
SELECT E.ENAME,D.DNAME
FROM EMP E FULL OUTER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO ;
```

=> returns all rows from emp & dept tables

A	ACCOUNTS	
B	RESEARCH	
C	SALES	
D	RESEARCH	
E	NULL	=> unmatched from emp
	NULL	OPERATIONS=> unmatched from dept

Displaying only unmatched records :-

left side table :-

```
SELECT E.ENAME,D.DNAME
FROM EMP E LEFT OUTER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO
WHERE D.DNAME IS NULL ;
```

E	NULL
---	------

right side table :-

```
SELECT E.ENAME,D.DNAME
FROM EMP E RIGHT OUTER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO
WHERE E.ENAME IS NULL ;
```

NULL	OPERATIONS
------	------------

both tables :-

```
SELECT E.ENAME,D.DNAME
```

```

FROM EMP E FULL OUTER JOIN DEPT D
  ON E.DEPTNO = D.DEPTNO
WHERE E.ENAME IS NULL
      OR
      D.DNAME IS NULL ;

E      NULL
NULL OPERATIONS

```

Question :-

EMP			PROJECTS		
EMPNO	ENAME	SAL	PROJID	PROJID	NAME DURATION CLIENT
1		100		100	
2		101		101	
3		NULL		102	

=> display employee details with project details ?

=> display employee details with project details and also display employees not assigned to any project ?

=> display only the projects where no employee assigned to it ?

SELF JOIN :-

=> joining a table to itself is called self join.

=> in self join a record in one table joined with another record of same table.

=> to perform self join the same table must be declared two times with different alias.

```

FROM EMP X JOIN EMP Y

```

EMP X			EMP Y		
EMPNO	ENAME	MGR	EMPNO	ENAME	MGR
7369	smith		7902	7369 smith	7902
7499	allen	7698	7499	allen	7698
7566	jones	7839	7566	jones	7839
7698	blake	7839	7698	blake	7839
7839	king	-----	7839	king	-----
7902	ford	7566	7902	ford	7566

=> display ENAME MGRNAME ?

```
SELECT X.ENAME,Y.ENAME AS MANAGER
FROM EMP X INNER JOIN EMP Y
ON X.MGR = Y.EMPNO ;
```

```
smith  ford
allen  blake
jones  king
blake  king
ford   jones
```

=> display employees reporting to blake ?

```
SELECT X.ENAME,Y.ENAME AS MANAGER
FROM EMP X INNER JOIN EMP Y
ON X.MGR = Y.EMPNO
WHERE Y.ENAME='BLAKE' ;
```

=> blake's manager name ?

```
SELECT X.ENAME,Y.ENAME AS MANAGER
FROM EMP X INNER JOIN EMP Y
ON X.MGR = Y.EMPNO
WHERE X.ENAME='BLAKE' ;
```

=> employees earning more than their managers ?

```
SELECT X.ENAME,X.SAL,
       Y.ENAME AS MANAGER,Y.SAL AS MGRSAL
FROM EMP X INNER JOIN EMP Y
ON X.MGR = Y.EMPNO
WHERE X.SAL > Y.SAL ;
```

=> employee having more experience than their manager ?

```
SELECT X.ENAME,X.HIREDATE,
       Y.ENAME AS MANAGER , Y.HIREDATE AS MGRHIRE
FROM EMP X INNER JOIN EMP Y
ON X.MGR = Y.EMPNO
WHERE X.HIREDATE < Y.HIREDATE ;
```

Question :-

```
TEAMS
ID    COUNTRY
1     IND
2     AUS
3     ENG
```

output :-

```
IND VS AUS
IND VS ENG
AUS VS ENG
```

17-MAY-24

GROUP BY clause :-

=====
=> GROUP BY clause is used for grouping rows based on one or more columns to calculate min,max,sum,avg,count for each group. For example to count no of employees for each dept first we need to group rows based on dept

```
EMP
EMPNO  ENAME      SAL  DEPTNO
1      A        4000  10  GROUP BY    10    7000
2      B        3000  20  =====> 20    9000
3      C        5000  30                30    5000
4      D        6000  20
5      E        3000  10
```

detailed data

summarized data

=> GROUP BY clause converts detailed data into summarized data which is useful for analysis.

syntax :-

```
SELECT columns
FROM tablename
[ WHERE cond]
GROUP BY <colname>
[HAVING cond]
```

[ORDER BY col ASC/DESC]

Execution :-

FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY

=> display dept wise total salary ?

```
SELECT DEPTNO,SUM(SAL)
FROM EMP
GROUP BY DEPTNO ;
```

10	8750
20	10875
30	9400

=> display job wise min sal,max sal,total sal,no of employees ?

```
SELECT JOB,MIN(SAL) AS MINSAL,
        MAX(SAL) AS MAXSAL,
        SUM(SAL) AS TOTSAL,
        COUNT(*) AS CNT
FROM EMP
GROUP BY JOB ;
```

CLERK	800	1300	4150	4	
SALESMAN		1250	1600	5600	4
ANALYST		3000	3000	6000	2
MANAGER		2450	2975	8275	3
PRESIDENT		5000	5000	5000	1

=> display year wise no of employees joined ?

```
SELECT TO_CHAR(HIREDATE,'YYYY') AS YEAR,
        COUNT(*)
FROM EMP
GROUP BY TO_CHAR(HIREDATE,'YYYY') ;
```

=> display day wise no of employees joined ?

```
SELECT TO_CHAR(HIREDATE,'DAY') AS YEAR,  
       COUNT(*)  
FROM EMP  
GROUP BY TO_CHAR(HIREDATE,'DAY');
```

=> display month wise no of employees joined in year 1981 ?

```
SELECT TO_CHAR(HIREDATE,'month') AS YEAR,  
       COUNT(*)  
FROM EMP  
WHERE TO_CHAR(HIREDATE,'YYYY') = 1981  
GROUP BY TO_CHAR(HIREDATE,'month');
```

=> find the departments having more than 3 employees ?

```
SELECT DEPTNO,COUNT(*)  
FROM EMP  
WHERE COUNT(*) > 3  
GROUP BY DEPTNO ; => ERROR
```

note :-

=> oracle cannot calculate dept wise count before group by and it can calculate only after group by , so apply the condition COUNT(*) > 3 after group by using HAVING clause.

```
SELECT DEPTNO,COUNT(*)  
FROM EMP  
GROUP BY DEPTNO  
HAVING COUNT(*) > 3 ;
```

WHERE VS HAVING :-

WHERE	HAVING
1 to select specific rows	to select specific groups
2 conditions applied before group by	conditions applied after group by

3 use where clause use having clause
if cond doesn't if cond contains group function
contain group function

`=> find southern states having more 5cr population ?

PERSONS
AADHARNO NAME AGE GENDER ADDR CITY STATE

```
SELECT STATE,COUNT(*)  
FROM PERSONS  
WHERE STATE IN ('AP','TS','KA','KL','TN')  
GROUP BY STATE  
HAVING COUNT(*) > 50000000
```

=> find the total sal paid to departments where deptno = 10,20 and total sal > 10000 ?

```
SELECT DEPTNO,SUM(SAL)  
FROM EMP  
WHERE DEPTNO IN (10,20)  
GROUP BY DEPTNO  
HAVING SUM(SAL) > 10000;
```

18-may-24

=> display dept wise and with in dept job wise total sal ?

SQL>BREAK ON DEPTNO SKIP 1

```
SQL> SELECT DEPTNO,JOB,SUM(SAL)  
FROM EMP  
GROUP BY DEPTNO , JOB  
ORDER BY DEPTNO ASC ;
```

10	CLERK	1300
	MANAGER	2450
	PRESIDENT	5000

20	ANALYST	6000
	CLERK	1900
	MANAGER	2975

30	CLERK	950
----	-------	-----

MANAGER 2850
SALESMAN 5600

=> display year wise and with in year quarter wise no of employees joined ?

1981	1	?
2	?	
3	?	
4	?	

```
SELECT TO_CHAR(HIREDATE,'YYYY') AS YEAR,
       TO_CHAR(HIREDATE,'Q') AS QRT,
       COUNT(*) AS CNT
FROM EMP
GROUP BY TO_CHAR(HIREDATE,'YYYY') , TO_CHAR(HIREDATE,'Q')
ORDER BY YEAR ASC ;
```

GROUP BY YEAR => INVALID
ORDER BY YEAR => VALID

NOTE :-

=> alias cannot be used in GROUP BY clause because group by clause is executed before select.

=> alias can be used in ORDER BY clause because order by clause is executed after select.

GROUP BY & JOIN :-

```
SELECT columns
FROM tab1 INNER JOIN tab2
  ON join cond
GROUP BY colname
ORDER BY col ASC/DESC ;
```

Ex :-

=> display dept wise total sal ? display dept names ?

```
SELECT D.DNAME,SUM(E.SAL)
FROM EMP E INNER JOIN DEPT D
```

ON E.DEPTNO = D.DEPTNO
GROUP BY D.DNAME ;

Question :-

1 display no of employees working under each manager ?

2

SALES
DATEID PRODID CUSTID QTY AMT
17-MAY-24 100 10 1 2000

PRODUCTS
PRODID PNAME PRICE CATEGORY
100 A 2000 ELECTRONICS

CUST
CUSTID NAME ADDR COUNTRY
10 K HYD IND

=> display year wise total amount ?
=> display category wise total amount ?
=> display country wise total amount ?

SUB-QUERIES / NESTED QUERIES :-

=> a query in another query is called sub-query or nested query
=> one query is called inner / child / sub query
=> other query is called outer / parent / main query
=> oracle executes inner query then it executes outer query
=> result of inner query is input to outer query
=> use sub-queries when where cond based on unknown value

Types of sub-queries :-

- 1 single row sub-queries
- 2 multi row sub-queries
- 3 co-related sub-queries
- 4 inline views

single row sub-queries :-

=> if inner query returns one value then it is called single row sub-query

```
SELECT columns
FROM tablename
WHERE colname OP (SELECT STATEMENT);
```

=> OP must be any relational operator like > >= < <= <>

Examples :-

=> employees earning more than blake ?

```
SELECT *
FROM EMP
WHERE SAL > (SELECT SAL FROM EMP WHERE ENAME='BLAKE');
```

=> employees who are senior to king ?

```
SELECT *
FROM EMP
WHERE HIREDATE < (SELECT HIREDATE FROM EMP WHERE ENAME='KING');
```

17-NOV-81

=> name of the employee earning max salary ?

```
SELECT ENAME
FROM EMP
WHERE SAL = (SELECT MAX(SAL) FROM EMP);
```

5000

=> name of the employee having max experience ?

```
SELECT ENAME
FROM EMP
WHERE HIREDATE = (SELECT MIN(HIREDATE) FROM EMP);
```

=> display 2nd max sal ?

SAL
5000
1000
3000
200
1500

```
SELECT MAX(SAL)
FROM EMP
WHERE SAL <> (SELECT MAX(SAL) FROM EMP) ;
```

o/p :- 3000

=> name of the employee earning 2nd max sal ?

```
SELECT ENAME,SAL
FROM EMP
WHERE SAL = ( SELECT MAX(SAL)
              FROM EMP
              WHERE SAL <> (SELECT MAX(SAL) FROM EMP) );
```

=> hike the employee sal having max experience by 10% ?

```
UPDATE EMP
SET SAL = SAL + (SAL*0.1)
WHERE HIREDATE = (SELECT MIN(HIREDATE) FROM EMP) ;
```

=> delete the employee having max experience ?

```
DELETE FROM EMP WHERE HIREDATE = (SELECT MIN(HIREDATE) FROM EMP) ;
```

20-MAY-24

multi row sub-queries :-

=> if inner query returns more than one value then it is called multi row sub-query

```
SELECT columns
FROM tablename
WHERE colname OP (SELECT STATEMENT) ;
```

=> OP must be IN , NOT IN,ANY,ALL

single	multi
=	IN
<>	NOT IN
>	>ANY >ALL
<	<ANY <ALL

=> employee names working at NEW YORK,CHICAGO locations ?

sub-query :-

```
SELECT ENAME
FROM EMP
WHERE DEPTNO IN (SELECT DEPTNO
                  FROM DEPT
                  WHERE LOC IN ('NEW YORK','CHICAGO')) ;
```

join :-

```
SELECT E.ENAME
FROM EMP E INNER JOIN DEPT D
      ON E.DEPTNO = D.DEPTNO
WHERE D.LOC IN ('NEW YORK','CHICAGO') ;
```

=> display ENAME DNAME working at NEW YORK,CHICAGO locations ?

JOIN :-

```
SELECT E.ENAME,D.DNAME
FROM EMP E INNER JOIN DEPT D
      ON E.DEPTNO = D.DEPTNO
WHERE D.LOC IN ('NEW YORK','CHICAGO') ;
```

SUB-QUERY :-

not possible

difference between sub-query and join ?

1 to display data from one table and condition based on another table
then we can use join or sub-query

2 to display data from two tables then use join operation

ANY & ALL operators :-

=> use ANY,ALL operators for > < comparision with multiple values

where x > any(1000,2000,3000) where x < any(1000,2000,3000)

if x = 800	false	if x = 800	true
1500	true	1500	true
4500	true	4500	false

where x > all(1000,2000,3000) where x < all(1000,2000,3000)

if x = 800	false	if x = 800	true
1500	false	1500	false
4500	true	4500	false

=> employees earning more than all managers ?

```
SELECT *
FROM EMP
WHERE SAL > ALL (SELECT SAL FROM EMP WHERE JOB='MANAGER');
```

2975
2850
2450

=> employees earning more than atleast one manager ?

```
SELECT *
FROM EMP
WHERE SAL > ANY (SELECT SAL FROM EMP WHERE JOB='MANAGER');
```

co-related sub-queries :-

=> if sub-query references values of outer query then it is called co-related sub-query

=> execution starts from outer query and inner query is executed no of times depends on no of rows return by outer query.

=> use co-related sub-query to execute sub-query for each row

ex :-

EMP

EMPNO	ENAME	SAL	DEPTNO
-------	-------	-----	--------

1	A	3000	10
---	---	------	----

2	B	4000	20
---	---	------	----

3	C	5000	30
---	---	------	----

4	D	6000	10
---	---	------	----

5	E	2000	20
---	---	------	----

=> employees earning more than avg sal of the organization ?

```
SELECT *  
FROM EMP  
WHERE SAL > (SELECT AVG(SAL) FROM EMP);
```

4000

=> employees earning more than avg sal of their dept ?

```
SELECT *  
FROM EMP X  
WHERE SAL > (SELECT AVG(SAL)  
             FROM EMP  
             WHERE DEPTNO = X.DEPTNO) ;
```

EMPNO	ENAME	SAL	DEPTNO
-------	-------	-----	--------

1	A	3000	10	3000 > (WHERE DEPTNO = 10)	4500	FALSE
---	---	------	----	----------------------------	------	-------

2	B	4000	20	4000 > (WHERE DEPTNO = 20)	3000	TRUE
---	---	------	----	----------------------------	------	------

3	C	5000	30	5000 > (WHERE DEPTNO = 30)	5000	FALSE
---	---	------	----	----------------------------	------	-------

4	D	6000	10	6000 > (WHERE DEPTNO= 10)	4500	TRUE
---	---	------	----	---------------------------	------	------

5	E	2000	20	2000 > (WHERE DEPTNO =20)	3000	FALSE
---	---	------	----	---------------------------	------	-------

=> employees who are earning max sal in their dept ?

```
SELECT *
FROM EMP X
WHERE SAL = (SELECT MAX(SAL)
              FROM EMP
              WHERE DEPTNO = X.DEPTNO) ;
```

1	A	3000	10	3000 = (6000)	FALSE
2	B	4000	20	4000 = (4000)	TRUE
3	C	5000	30	5000 = (5000)	TRUE
4	D	6000	10	6000 = (6000)	TRUE
5	E	2000	20	2000 = (4000)	FALSE

=> display top 3 max salaries ?

```
SELECT A.SAL
FROM EMP A
WHERE 3 > (SELECT COUNT(B.SAL)
           FROM EMP B
           WHERE A.SAL < B.SAL)
```

EMP A		EMP B		
SAL	SAL			
5000		5000	3 > (0)	TRUE
1000		1000	3 > (4)	FALSE
3000		3000	3 > (1)	TRUE
3000		3000	3 > (1)	TRUE
2975		2975	3 > (3)	FALSE

```
SELECT DISTINCT A.SAL
FROM EMP A
WHERE 3 > (SELECT COUNT(DISTINCT B.SAL)
           FROM EMP B
           WHERE A.SAL < B.SAL)
ORDER BY SAL DESC ;
```

EMP A		EMP B		
SAL	SAL			
5000		5000	3 > (0)	TRUE
1000		1000	3 > (3)	FALSE

3000	3000	3 > (1) TRUE
3000	3000	3 > (1) TRUE
2975	2975	3 > (2) TRUE

21-may-24

=> display 3rd max salary ?

```
SELECT DISTINCT A.SAL
FROM EMP A
WHERE (3-1) = (SELECT COUNT(DISTINCT B.SAL)
               FROM EMP B
               WHERE A.SAL < B.SAL)
ORDER BY SAL DESC ;
```

=> display Nth max salary ?

```
SELECT DISTINCT A.SAL
FROM EMP A
WHERE (&N-1) = (SELECT COUNT(DISTINCT B.SAL)
               FROM EMP B
               WHERE A.SAL < B.SAL)
ORDER BY SAL DESC ;
```

ROWID :-

=> returns physical address of a row i.e. where the record is stored in memory

=> ROWID is called psuedo column because it is not a column but acts like a column

ex :- SELECT ROWID,EMPNO,ENAME,SAL FROM EMP ;

=> rowids are used to delete duplicate records

ex :-

EMP44			
EMPNO		ENAME	SAL ROWID
1	A	5000	AAA
2	B	6000	AAB
3	C	7000	AAC
1	A	5000	AAD
2	B	6000	AAE

```
DELETE FROM EMP44 WHERE ROWID LIKE '%AAD' ;
```

```
DELETE FROM EMP44 WHERE ROWID LIKE '%AAE' ;
```

=> deleting duplicates row-by-row is difficult if table contains lakhs of duplicate rows , so execute the following query to delete all duplicate rows from the table.

```
DELETE
FROM EMP44 X
WHERE ROWID <> (SELECT MIN(ROWID)
                FROM EMP44
                WHERE EMPNO = X.EMPNO
                AND
                ENAME = X.ENAME
                AND
                SAL = X.SAL);
```

EMP44						
	EMPNO		ENAME	SAL	ROWID	
1	A	5000	AAA	<> (AAA)		FALSE
2	B	6000	AAB	<> (AAB)		FALSE
3	C	7000	AAC	<> (AAC)	FALSE	
1	A	5000	AAD	<> (AAA)		TRUE
2	B	6000	AAE	<> (AAB)		TRUE

INLINE views :-

=> sub-queries in FROM clause are called inline views

```
SELECT columns
FROM (select statement)
WHERE cond ;
```

=> sub-query output acts like a table for outer query

=> inline views are used in following scenarios

- 1 to control order of execution of clause
- 2 to join two query outputs

controlling order of execution of clauses :-

default order :-

```
FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY
```

=> use INLINE views to control this order of execution

```
SELECT
FROM
WHERE
ORDER BY
```

```
SELECT
FROM (SELECT
      FROM
      ORDER BY )
WHERE
```

example 1 :-

=> display ranks of the employees based on sal and highest paid should get 1st rank ?

```
SELECT empno,ename,sal,
       dense_rank() over (order by sal desc) as rnk
FROM emp ;
```

above query returns ranks of all the employees , but to display top 5 employees

```
SELECT empno,ename,sal,
       dense_rank() over (order by sal desc) as rnk
FROM emp
WHERE rnk <= 5 ; => error
```

column alias cannot be used in where clause because where clause is executed before select , to overcome this problem use INLINE views

```

SELECT *
FROM (SELECT empno,ename,sal,
             dense_rank() over (order by sal desc) as rnk
      FROM emp) E
WHERE rnk <= 5 ;

```

```

SELECT *
FROM E
WHERE rnk<=5 ;

```

=> top 5 max salaries ?

```

SELECT DISTINCT sal
FROM (SELECT sal,
             dense_rank() over (order by sal desc) as rnk
      FROM emp) E
WHERE rnk <= 5 ;

```

ROWNUM :-

=> rownum returns record number for the records return by select query
=> rownum is also a psuedo column

psuedo columns :-

- 1 rowid
- 2 rownum
- 3 sysdate
- 4 level

ex :-

```
SQL>SELECT rownum,empno,ename,sal FROM emp ;
```

```

9      7839  KING  5000

```

```
SQL>SELECT rownum,empno,ename,sal FROM emp WHERE sal>=3000;
```

```

2      7839  KING  5000

```

note :-

=> rownum is not based on table and it is based on query output
if query output changes rownum also changes

=> rownum is useful when fetching records from table based on record number

ex :-

=> display first 5 rows from emp table ?

```
SELECT empno,ename,sal
FROM emp
WHERE rownum <= 5 ;
```

=> display 5th record ?

```
SELECT empno,ename,sal
FROM emp
WHERE rownum = 5 ; => no rows
```

NOTE :- in where condition with rownum = > >= operators will not work
only < <= operators will work, to overcome this problem use INLINE views

```
SELECT *
FROM (SELECT ROWNUM AS RNO,EMPNO,ENAME,SAL FROM EMP) E
WHERE RNO = 5 ;
```

```
WHERE RNO IN (5,7,10) ;
```

```
WHERE RNO BETWEEN 5 AND 10 ;
```

```
WHERE MOD(RNO,2) = 0 ;
```

=> display last 3 rows from emp ?

```
SELECT *
FROM (SELECT ROWNUM AS RNO,EMPNO,ENAME,SAL FROM EMP) E
WHERE RNO >= (SELECT COUNT(*)-2 FROM EMP) ;
```

=>

TRANSACTIONS

TRID	TTYPE	TDATE	TAMT	ACCNO
1	W	?	2000	100

display latest 5 transactions of particular customer ?

```
SELECT *
FROM (SELECT *
      FROM TRANSACTIONS
      WHERE ACCNO = 100
      ORDER BY TDATE DESC) E
WHERE ROWNUM<=5 ;
```

22-MAY
21-
20-
19-
18-

simple select stmt
where clause
order by
distinct
functions
group by
joins
sub-queries

=====

DATABASE TRANSACTIONS :-

=> a transaction is a unit of work that contains one or more dmls
and must be saved as a whole or must be cancelled as a whole.

ex :- money transfer

acct1-----1000----->acct2

update1
(bal=bal-1000)

update2
(bal=bal+1000)

successful	failed	INVALID
failed	successful	INVALID
successful	successful	VALID
failed	failed	VALID

=> every txn must guarantee a property called atomocity i.e. all or none
if txn contains multiple operations then if all are successful then
it must be saved , if one of the operation fails then entire txn must be
cancelled.

=> the following commands provided by oracle to handle transactions called
TCL commands.

- 1 COMMIT => to save txn
- 2 ROLLBACK => to cancel txn
- 3 SAVEPOINT => to cancel part of the txn

=> every txn has a begin point and an end point

=>a txn begins automatically whenever user submits dml command to oracle

=> a txn ends with any of the following commands

- 1 commit / rollback
- 2 ddl command (txn ends with commit)

ddl command = ddl command + commit

Example 1 :-

```
create table a(a number(2)); => commit
insert into a values(10);    => txn begins T1
insert into a values(20);
insert into a values(30);
insert into a values(40);
rollback ;                  => txn ends
```

if txn ends with rollback then operations are cancelled

Example 2 :-

```
create table a(a number(2)); => commit
insert into a values(10);      => txn starts t1
insert into a values(20);
create table b(b number(2)); => commit
insert into a values(30);      => txn begins t2
insert into a values(40);
rollback ;                     => txn ends
```

```
create table a => saved
insert 10,20   => saved
create table b => saved
insert 30,40   => cancelled
```

SAVEPOINT :-

=> we can declare savepoint and we can rollback upto the savepoint
=> using savepoint we can cancel part of the txn

```
CREATE TABLE A(A NUMBER(2));
INSERT INTO A VALUES(10);
INSERT INTO A VALUES(20);
SAVEPOINT SP1;
INSERT INTO A VALUES(30);
INSERT INTO A VALUES(40);
SAVEPOINT SP2;
INSERT INTO A VALUES(50);
INSERT INTO A VALUES(60);
ROLLBACK TO SP1;
```

```
SELECT * FROM A ;
```

```
10
20
```

23-MAY-24

LOCKING :-

=> accessing same data by no of users at the sametime is called concurrent access
when data accessed concurrently users encounters following problems

- 1 dirty read
- 2 lost update
- 3 phantom read
- 4 non repeatable read

=> to overcome these problems every db system supports a mechanism called locking mechanism

=> locks are 2 types / modes

- 1 SHARED LOCK
- 2 EXCLUSIVE LOCK

=> oracle will apply shared lock whenever we try to read data i.e. whenever user submits select stmt

=> oracle will apply exclusive lock whenever we try to modify the data. whenever user submits DML commands.

	S	X
S	YES	YES
X	YES	NO

Example :-

SYSTEM	BATCH32
1 UPDATE BATCH32.EMP SET SAL=1000 WHERE EMPNO=7369;	2 UPDATE EMP SET SAL = 2000 WHERE EMPNO = 7369 ;
updated + locked	-----wait-----
3 COMMIT; (LOCK RELEASES)	4 1 row updated

DEADLOCK :-

=> deadlock is the situation where two users mutually waits for one another
if deadlock occurs then oracle returns error so that one transaction
can be cancelled and another can be continued.

ex :-

SYSTEM	BATCH32
1 UPDATE BATCH32.EMP SET SAL=1000 WHERE EMPNO=7369; updated + locked	2 UPDATE EMP SET SAL = 2000 WHERE EMPNO = 7499; updated + locked
3 UPDATE BATCH32.EMP SET SAL=1000 WHERE EMPNO=7499; -----WAIT-----	4 UPDATE EMP SET SAL = 2000 WHERE EMPNO = 7369; -----WAIT-----
-----DEADLOCK-----	

5 ERROR :- deadlock detected

6 ROLLBACK;

TABLE LEVEL LOCKING :-

=> complete table is locked i.e. all the records of the table are locked

LOCK TABLE <TABNAME> IN <MODE> ;

EX :-

SQL>LOCK TABLE EMP IN EXCLUSIVE MODE ;

=====

DB OBJECTS :-

-
- 1 TABLES
 - 2 SEQUENCES
 - 3 SYNONYMS
 - 4 VIEWS
 - 5 INDEXES

SEQUENCES :-

=> sequence is a db object created to generate sequence numbers for primary key columns

=> used to auto increment column values

syn :-

```
CREATE SEQUENCE <NAME>
[START WITH <VALUE>]
[INCREMENT BY <VALUE>]
[MAXVALUE <VALUE>] ;
```

Ex :-

```
CREATE SEQUENCE S1
START WITH 1
INCREMENT BY 1
MAXVALUE 5 ;
```

using sequence :-

```
CREATE TABLE STUDENT(SID NUMBER(2), SNAME VARCHAR2(10));
```

use sequence s1 to generate values for SID ?

```
INSERT INTO STUDENT VALUES(S1.NEXTVAL , 'A');
INSERT INTO STUDENT VALUES(S1.NEXTVAL , 'B');
INSERT INTO STUDENT VALUES(S1.NEXTVAL , 'C');
INSERT INTO STUDENT VALUES(S1.NEXTVAL , 'D');
INSERT INTO STUDENT VALUES(S1.NEXTVAL , 'E');
```

```
SELECT * FROM STUDENT ;
```

SID	SNAME
1	A
2	B
3	C
4	D
5	E

Example 2 :-

```
CREATE SEQUENCE S2
START WITH 100
INCREMENT BY 1
MAXVALUE 999;
```

=> use sequence S2 to generate empno ?

```
SQL>UPDATE EMP SET EMPNO = S2.NEXTVAL ;
```

Example 3 :-

```
CREATE SEQUENCE S3
START WITH 1
INCREMENT BY 1
MAXVALUE 9999;
```

INVOICE		
INVNO	INVDT	INVAMT
NIT/0524/1	?	?
NIT/0524/2	?	?

```
CREATE TABLE INVOICE
(
  INVNO    VARCHAR2(20) ,
  INVDT    DATE,
  INVAMT    NUMBER(5)
);
```

```
INSERT INTO INVOICE
VALUES('NIT/'||TO_CHAR(SYSDATE,'MMYY')||'/'||S3.NEXTVAL,SYSDATE,500);
```

```
SELECT * FROM INVOICE ;
```

INVNO	INVDT	INVAMT
NIT/0524/1	23-MAY-24	500
NIT/0524/2	23-MAY-24	500
NIT/0524/3	23-MAY-24	500
NIT/0524/4	23-MAY-24	500
NIT/0524/5	23-MAY-24	500

Dropping :-

```
SQL>DROP SEQUENCE S1 ;
```

SYNONYMS :-

=> a synonym is another name or alternative name to a table

=> if tablename is lengthy then we can give a simple and short name to the table called synonym and instead of using tablename we can use synonym name in SELECT / INSERT / UPDATE / DELETE queries.

```
syn :- CREATE SYNONYM <NAME> FOR <TABNAME> ;
```

```
ex :- CREATE SYNONYM E FOR EMP ;
```

=> after creating synonym instead of using tablename use synonym name

```
1 SELECT * FROM E ;
```

```
2 UPDATE E SET SAL = 2000 WHERE EMPNO = 7844;
```

Question :-

```
SQL>CREATE SYNONYM E FOR EMP ;
```

```
SQL> SELECT * FROM EMP E ;
```

```
SQL>RENAME EMP TO E ;
```

difference between synonym and alias ?

SYNONYM

ALIAS

- | | | |
|---|--|------------------|
| 1 | stored in db | not stored in db |
| 2 | permanent | not permanent |
| 3 | scope of the synonym
is upto the schema | upto the query |

Dropping synonym :-

SQL>DROP SYNONYM E ;

=====

24-may-24

PL/SQL

=====

PL => procedural language

ORACLE

NON-PROC

PROC

SQL

PL/SQL

(commands)

(blocks)

Features :-

improves performance :-

=> In pl/sql , sql commands can be grouped into one block and we submit that block to oracle, so in pl/sql no of requests and response between user and oracle are reduced and performance is improved.

supports conditional statements :-

=> pl/sql supports conditional statements like IF-ELSE , so in pl/sql we can execute sql commands based on conditions.

supports loops :-

=> pl/sql supports looping statements like while,for , so with the help of loops we can execute commands repeatedly multiple times.

supports error handling :-

=> pl/sql supports error handling , In pl/sql if any statement causes error then we can handle that error and we can display our own simple and user friendly message.

supports reusability :-

=> pl/sql programs can be stored in db , so applications which are connected to db can reuse pl/sql programs.

supports security :-

=> because these programs are stored in db , so only authorized users can execute these programs

=> PL/SQL blocks are 2 types

- 1 Anonymous Blocks
- 2 Named Blocks
 - procedures
 - functions
 - packages
 - triggers

Anonymous Blocks :-

=> a program without name is called anonymous block

```
DECLARE
    declaration-part;    optional
BEGIN
    execution-part;
END;
/    => compilation & execution starts
```

How to print messages :-

```
DBMS_OUTPUT.PUT_LINE(message);
```

PACKAGE PROCEDURE

=> by default messages are not send to output , to send messages to output execute the following command.

```
SQL>SET SERVEROUTPUT ON
```

How to write programs :-

1 EDITORs

2 IDEs

	EDITOR	IDE
coding	YES	YES
compilation	NO	YES
execution	NO	YES
debugging	NO	YES
example	NOTEPAD	SQL DEVELOPER

using NOTEPAD :-

=> open notepad and enter following code

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('HELLO');
END;
/
```

=> save the program in a file as D:\PLSQL\PROG1.SQL"

=> go to sqlplus compile and run the program as follows

```
SQL>@D:\PLSQL\PROG1.SQL
```

Datatypes in PL/SQL :-

- 1 NUMBER(P) / NUMBER(P,S)
- 2 CHAR/VARCHAR2/LONG/CLOB
- 3 NCHAR/NVARCHAR2/NCLOB
- 4 DATE / TIMESTAMP
- 5 BFILE / BLOB
- 6 BOOLEAN

1 TO 5 => allowed in sql, pl/sql

6 => allowed only in pl/sql

Declaring variable :-

```
variablename datatype(size) ;
```

ex :-

```
x  number(4);
s  varchar2(20);
d  date;
b  boolean;
```

Assigning value to variable :-

```
variablename := value ;
```

`:=` assignment

`=` comparison

```
ex :-      x := 100;
           s := 'abc';
           d := sysdate;
           b := TRUE ;
```

=> write a prog to add two numbers ?

```
DECLARE
  a  NUMBER(3);
  b  NUMBER(3);
  c  NUMBER(3);
BEGIN
  a := &a;
  b := &b;
  c := a+b;
  DBMS_OUTPUT.PUT_LINE(c);
END;
/
```

=> write a prog to input date and print day of the week ?

```
DECLARE
  d  DATE ;
BEGIN
  d := '&date' ;
  DBMS_OUTPUT.PUT_LINE(TO_CHAR(d,'day'));
END;
/
```

DB PROGRAMMING WITH PL/SQL :-

=> From pl/sql to work with db execute sql commands from pl/sql program

=> the following commands can be executed from pl/sql program.

- 1 DML (insert,update,delete,merge)
- 2 DQL (select)
- 3 TCL (commit,rollback,savepoint)

SELECT stmt syntax :-

SELECT columns INTO variables
FROM tabname
WHERE condition ;

ex :-

SELECT ENAME,SAL INTO N,S
FROM EMP
WHERE EMPNO = 7844 ;

=> write a prog to input empno and print name & salary ?

```
DECLARE
    vempno    NUMBER(4);
    vename    VARCHAR2(10);
    vsal      NUMBER(7,2);
BEGIN
    vempno := &empno;
    SELECT ename,sal INTO vename,vsal
    FROM emp
    WHERE empno = vempno;
    DBMS_OUTPUT.PUT_LINE(vename||' '||vsal);
END;
/
```

=> write a prog to input empno and calculate and print experience ?

```
DECLARE
    vempno    NUMBER(4);
    vhire     DATE;
    vexpr     NUMBER(2);
BEGIN
    vempno := &empno;
    SELECT hiredate INTO vhire
    FROM emp
    WHERE empno = vempno;
    vexpr := (SYSDATE-vhire)/365 ;
    DBMS_OUTPUT.PUT_LINE(' experience = '||vexpr||' '||'years');
END;
/
```

experience = ?? years

27-MAY-24

=> write a prog to input empno and calculate total sal ?

TOTAL SAL =SAL + COMM

```
DECLARE
  veno NUMBER(4);
  vsal  NUMBER(7,2);
  vcomm NUMBER(7,2);
  vtotsal NUMBER(8,2);
BEGIN
  veno := &empno;
  SELECT sal,comm INTO vsal,vcomm
  FROM emp
  WHERE empno = veno;
  vtotsal := vsal + NVL(vcomm,0);
  DBMS_OUTPUT.PUT_LINE('Total Sal = '||vtotsal);
END;
/
```

Conditional statements :-

- 1 IF-ELSE
- 2 MULTI IF
- 3 NESTED IF

IF-ELSE :-

```
IF COND THEN
  statements;
ELSE
  statements;
END IF;
```

MULTI IF :-

```

IF COND1 THEN
    statements;
ELSIF COND2 THEN
    statements;
ELSIF COND3 THEN
    statements;
ELSE
    statements;
END IF;

```

NESTED IF :-

```

IF COND THEN
    IF COND THEN
        statements;
    ELSE
        statements;
    END IF;
ELSE
    statements;
END IF;

```

=> write a prog to input empno and increment sal by specific amount after increment if sal exceeds 5000 then cancel that increment ?

```

DECLARE
    veno    NUMBER(4);
    vamt    NUMBER(5);
    vsal    NUMBER(7,2);
BEGIN
    veno := &empno;
    vamt := &amount;
    UPDATE emp SET sal = sal + vamt WHERE empno = veno ;
    SELECT sal INTO vsal FROM emp WHERE empno = veno;
    IF vsal > 5000 THEN
        ROLLBACK;
    ELSE
        COMMIT;
    END IF;
END;
/

```

=> write a prog to input empno and increment sal as follows ?

IF JOB=CLERK	incr sal by 10%
MANAGER	15%
SALESMAN	20%
others	5%

```
DECLARE
  veno  NUMBER(4);
  vjob   VARCHAR2(10);
  vpct   NUMBER(2);
BEGIN
  veno := &empno;
  SELECT job INTO vjob FROM emp WHERE empno = veno;
  IF vjob='CLERK' THEN
    vpct := 10;
  ELSIF vjob='MANAGER' THEN
    vpct := 15;
  ELSIF vjob='SALESMAN' THEN
    vpct := 20;
  ELSE
    vpct := 5;
  END IF;
  UPDATE emp SET sal = sal + (sal*vpct/100) WHERE empno = veno;
  COMMIT;
END;
/
```

=> write a prog to process bank transaction (w/d) ?

ACCOUNTS		
ACCNO	ACTYPE	BAL
100 S	10000	
101 S	20000	

```
DECLARE
  vacno  NUMBER(4);
  vtype  CHAR(1);
  vamt   NUMBER(5);
  vbal   NUMBER(8);
BEGIN
  vacno := &acno;
  vtype := '&type';
```

```

vamt := &amount;
IF vtype='W' THEN
    SELECT bal INTO vbal FROM accounts WHERE accno = vacno;
    IF vamt > vbal THEN
        DBMS_OUTPUT.PUT_LINE('insufficient balance');
    ELSE
        UPDATE accounts SET bal = bal - vamt WHERE accno = vacno;
    END IF;
ELSIF vtype='D' THEN
    UPDATE accounts SET bal = bal + vamt WHERE accno = vacno;
ELSE
    DBMS_OUTPUT.PUT_LINE('invalid transaction type');
END IF;
COMMIT;
END;
/

```

=> write a prog to process money transfer ?

LOOPS :-

- 1 simple loop
- 2 while loop
- 3 for loop

simple loop :-

```

LOOP
    statements;
    EXIT WHEN COND;
END LOOP;

```

if cond = false loop continues

if cond = true loop terminates

WHILE loop :-

```

WHILE(cond)
LOOP
    statements;

```

END LOOP;

if cond = true loop continues
if cond = false loop terminates

FOR loop :-

```
FOR <var> IN <low>..<upp>
LOOP
    statements;
END LOOP;
```

Ex :-

```
FOR x IN 1..10
LOOP
    statements;
END LOOP;
```

=> loop variable "x" declared implicitly as number
=> by default "x" value incremented by 1

=> write a prog to print numbers from 1 to 20 ?

using simple loop :-

```
DECLARE
    x  NUMBER(2) := 1;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE(x);
        x := x+1;
        EXIT WHEN x > 20;
    END LOOP;
END;
/
```

using while loop :-

```
DECLARE
```

```

    x NUMBER(2) := 1;
BEGIN
    WHILE(x<=20)
    LOOP
        DBMS_OUTPUT.PUT_LINE(x);
        x := x+1;
    END LOOP;
END;
/

```

using for loop :-

```

BEGIN
    FOR x IN 1..20                for(x=1;x<=20;x++)
    LOOP
        DBMS_OUTPUT.PUT_LINE(x);
    END LOOP;
END;
/

```

```

BEGIN
    FOR x IN 20..1                for(x=20;x<=1;x++)
    LOOP
        DBMS_OUTPUT.PUT_LINE(x);
    END LOOP;
END;
/

```

```

BEGIN
    FOR x IN REVERSE 1..20
    LOOP
        DBMS_OUTPUT.PUT_LINE(x);
    END LOOP;
END;
/

```

=> write a prog to print 2024 calendar ?

01-jan-24 ?

02-jan-24 ?

31-dec-24 ?

```

DECLARE
    d1    DATE;
    d2    DATE;
BEGIN
    d1 := '01-JAN-24';
    d2 := '31-DEC-24';
    WHILE(d1<=d2)
    LOOP
        DBMS_OUTPUT.PUT_LINE(d1||'   '||TO_CHAR(d1,'day'));
        d1 := d1+1;
    END LOOP;
END;
/

```

```

DECLARE
    d1    DATE;
    d2    DATE;
BEGIN
    d1 := '01-JAN-24';
    d2 := '31-DEC-24';
    d1 := NEXT_DAY(d1,'sunday');
    WHILE(d1<=d2)
    LOOP
        DBMS_OUTPUT.PUT_LINE(d1||'   '||TO_CHAR(d1,'day'));
        d1 := d1+7;
    END LOOP;
END;
/

```

28-may-24

=> write a prog to input string and print following pattern ?

INPUT :- ORACLE

OUTPUT :-

O
R
A
C
L

E

```
DECLARE
  s1 VARCHAR2(20);
BEGIN
  s1 := '&string';
  FOR x IN 1..LENGTH(s1)
  LOOP
    DBMS_OUTPUT.PUT_LINE(SUBSTR(s1,x,1));
  END LOOP;
END;
/
```

=> write a prog to input string and print following pattern ?

INPUT :- ORACLE

OUTPUT :-

O
OR
ORA
ORAC
ORACL
ORACLE

```
DECLARE
  s1 VARCHAR2(20);
BEGIN
  s1 := '&string';
  FOR x IN 1..LENGTH(s1)
  LOOP
    DBMS_OUTPUT.PUT_LINE(SUBSTR(s1,1,x));
  END LOOP;
END;
/
```

=> write a prog to input string print reverse of that string ?

INPUT :- ORACLE

OUTPUT :- ELCARO

```

DECLARE
  s1 VARCHAR2(20);
  s2 VARCHAR2(20);
BEGIN
  s1 := '&string';
  FOR x IN 1..LENGTH(s1)
  LOOP
    s2 := s2||SUBSTR(s1,-x,1);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(s2);
  IF s1 = s2 THEN
    DBMS_OUTPUT.PUT_LINE('palindrome');
  ELSE
    DBMS_OUTPUT.PUT_LINE('not a palindrome');
  END IF;
END;
/

```

Reference Types :-

1 %TYPE

2 %ROWTYPE

%TYPE :-

=> used to refer column datatype & size

ex :- vname emp.ename%TYPE;

=> whatever datatype and size declared for column ename the same type and size assigned to variable vname.

=> %TYPE reduces complexity

%ROWTYPE :-

=> used to refer record type

ex :- e emp%ROWTYPE;

=> a row from emp table can be assigned to variable "e"

```
SELECT * INTO e
FROM emp
WHERE empno = 7844;
```

e	empno	ename	job	mgr	hiredate	sal	comm	deptno
	7844	turner	salesman			1500	0	30

=> e.sal => 1500
e.job => salesman

=> Adv of %rowtype is it reduces no of variables required in the program and reduces complexity.

example :-

STUDENT					
SNO	SNAME		S1	S2	S3
1	A	80	90	70	
2	B	30	60	50	

RESULT			
SNO	TOTAL	AVG	RESULT

=> write a prog to input sno and calculate total ,avg,result and insert into result table ?

```
DECLARE
    vsno student.sno%TYPE;
    s student%ROWTYPE;
    r result%ROWTYPE;
BEGIN
    vsno := &sno;
    SELECT * INTO s FROM student WHERE sno = vsno ;
    r.total := s.s1 + s.s2 + s.s3;
    r.avg := r.total/3;
    IF s.s1>=35 AND s.s2>=35 AND s.s3>=35 THEN
        r.result := 'PASS';
    ELSE
        r.result := 'FAIL';
    END IF;
    INSERT INTO RESULT VALUES(vsno,r.total,r.avg,r.result);
```

```
        COMMIT;
    END;
/
```

s

sno	sname	s1	s2	s3
1	A	80	90	70

r

sno	total	avg	result
	240	80	PASS

CURSOR :-

=> cursors are used to access multiple rows in pl/sql program.

=> using cursor we can access row-by-row in pl/sql program.

=> follow below steps to use cursor

- 1 declare
- 2 open
- 3 fetch
- 4 close

Declaring cursor :-

```
CURSOR <name> IS SELECT STATEMENT ;
```

ex :-

```
CURSOR C1 IS SELECT ENAME,SAL FROM EMP ;
```

Opening :-

```
OPEN <cursor-name>;
```

```
OPEN C1 ;
```

=> select stmt submitted to oracle

=> oracle executes query and data returned by query is copied to cursor

Fetching record from cursor :-

=> "FETCH" statement is used to fetch record from cursor

syn :- FETCH <cursor-name> INTO <variables>;

ex :- FETCH C1 INTO vename,vsal;

=> a fetch stmt fetched one row at a time but to process multiple rows
fetch stmt should be executed multiple times. so fetch stmt should be
inside a loop.

Closing cursor :-

 CLOSE <cursor-name>;

ex :- CLOSE C1

Attributes :-

%FOUND => returns true if fetch successful otherwise returns false

%NOTFOUND => returns true if fetch unsuccessful otherwise returns true

%ROWCOUNT => returns no of rows fetched successfully

C1%FOUND

C1%NOTFOUND

C`1%ROWCOUNT

=> write a prog to print all employee names and experience ?

DECLARE

 CURSOR C1 IS SELECT ENAME,HIREDATE FROM EMP ;

 vename emp.ename%TYPE;

 vdoj emp.hiredate%TYPE;

 vexpr NUMBER(2);

BEGIN

 OPEN C1;

 FETCH C1 INTO vename,vdoj;

 WHILE(C1%FOUND)

 LOOP

```

        vexpr := (SYSDATE-vdoj)/365 ;
        DBMS_OUTPUT.PUT_LINE(vename||' '||vexpr);
        FETCH C1 INTO vename,vdoj;
    END LOOP;
    CLOSE C1;
END;
/

```

29-may-24

CURSOR FOR LOOP / FOR LOOP CURSOR :-

```

FOR <var> IN <cursor-name>
LOOP
    statements;
END LOOP;

```

EX :-

```

FOR r IN C1
LOOP
    statements;
END LOOP;

```

=> for loop is executed no of times depends on no of rows in cursor
=> adv of for loop is opening cursor,fetching records and closing cursor is not required and all these operations performed implicitly.
=> everytime for loop executes a record is fetches from cursor and assigned to variable "r".
=> "r" is also declared implicitly as rowtype.

Example :-

```

DECLARE
    CURSOR C1 IS SELECT ENAME,SAL FROM EMP ;
BEGIN
    FOR R IN C1
    LOOP
        DBMS_OUTPUT.PUT_LINE(R.ENAME||' '||R.SAL);
    END LOOP;
END;
/

```

=> write a prog to calculate total sal without using sum function ?

```
DECLARE
  CURSOR C1 IS SELECT SAL FROM EMP ;
  t  NUMBER := 0;
BEGIN
  FOR R IN C1
  LOOP
    t:= t + R.SAL;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(t);
END;
/
```

=> write a prog to calculate max sal without using max function ?

```
DECLARE
  CURSOR C1 IS SELECT SAL FROM EMP ;
  m  NUMBER := 0;
BEGIN
  FOR R IN C1
  LOOP
    IF R.SAL > m THEN
      m := R.SAL;
    END IF;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(m);
END;
/
```

```
DECLARE
  CURSOR C1 IS SELECT SAL FROM EMP ORDER BY SAL DESC ;
  vsal NUMBER(8);
BEGIN
  OPEN C1;
  FETCH C1 INTO vsal;
  DBMS_OUTPUT.PUT_LINE(vsal);
  CLOSE C1;
END;
/
```

=> write a prog to print min sal ?

=> write a prog to calculate all the students total,avg,result and insert into result table ?

STUDENT				
SNO	SNAME	S1	S2	S3
1	A	80	90	70
2	B	30	60	50

RESULT			
SNO	TOTAL	AVG	RESULT

=> write a prog to input sno and calculate total ,avg,result and insert into result table ?

```
DECLARE
    CURSOR C1 IS SELECT * FROM STUDENT ;
    r RESULT%ROWTYPE;
BEGIN
    FOR s IN C1
    LOOP
        r.total := s.s1 + s.s2 + s.s3;
        r.avg := r.total/3;
        IF s.s1>=35 AND s.s2>=35 AND s.s3>=35 THEN
            r.result := 'FAIL';
        ELSE
            r.result := 'PASS';
        END IF;
        INSERT INTO RESULT VALUES(s.sno,r.total,r.avg,r.result);
    END LOOP;
    COMMIT;
END;
/
```

=====

ERROR HANDLING / EXCEPTION HANDLING :-

- 1 syntax errors
- 2 logical errors
- 3 runtime errors (exceptions)

=> errors raised during program execution are called runtime errors

ex :- X NUMBER(3);

X := &X ; => 10000 ; => runtime error

=> if statement causes runtime error then program execution is terminated and oracle displays error message, to replace system generated message with our own simple and user friendly message then we need to handle runtime error.

=> to handle runtime error include a block called exception block

```
DECLARE
    declaration-part;
BEGIN
    execution-part;    => stmts causes exception
EXCEPTION
    error handling - part; => stmts handles exception
END;
/
```

=> exceptions are 2 types

1 system defined

2 user defined

system defined exceptions :-

1 ZERO_DIVIDE => raised when we try to divide a number with 0

2 VALUE_ERROR => raised when size or type mismatch

3 NO_DATA_FOUND => raised when data not found in the table

4 DUP_VAL_ON_INDEX => raised when we try to insert duplicate value into pk column

Example 1 :-

```
DECLARE
    a NUMBER(3);
    b NUMBER(3);
    c NUMBER(3);
BEGIN
    a := &a;
    b := &b;
```

```

c := a/b;
DBMS_OUTPUT.PUT_LINE(c);
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('divisor cannot be zero');
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('value exceeding size');
END;
/

```

Example 2 :-

=> wap to input empno and print name & salary ?

```

DECLARE
    veno      emp.empno%TYPE;
    vename    emp.ename%TYPE;
    vsal      emp.sal%TYPE;
BEGIN
    veno := &empno;
    SELECT ename,sal INTO vename,vsal FROM emp WHERE empno = veno;
    DBMS_OUTPUT.PUT_LINE(vename||' '||vsal);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('invalid empno');
END;
/

```

user defined exceptions :-

- => exceptions raised by user are called user defined exceptions
- => user can raise exception to stop program execution based on condition
- => user can raise exception by using RAISE_APPLICATION_ERROR

```
RAISE_APPLICATION_ERROR(error code,error msg);
```

error code => -20001 to -20999

ex :-

=> write a prog to increment employee sal but sunday updates are not allowed ?

```

DECLARE
    veno emp.empno%TYPE;
    vamt NUMBER(5);
BEGIN
    veno := &empno;
    vamt := &amount;
    IF TO_CHAR(sysdate,'dy')='sun' THEN
        RAISE_APPLICATION_ERROR(-20001,'sunday not allowed');
    END IF;
    UPDATE EMP SET SAL = SAL + vamt WHERE EMPNO = veno;
    COMMIT;
END;
/

```

=====

=

30-may-24

Named PL/SQL Blocks :-

- 1 procedures
- 2 functions
- 3 packages
- 4 triggers

SUB-PROGRAMS :-

- 1 procedures
- 2 functions
- 3 packages

Advantages :-

- 1 modular programming :-

=> with help of proc & func a big pl/sql program can be divided into small modules

- 2 reusability :-

=> proc & func are stored in db , so applications which are connected to db
can reuse proc & func.

3 security :-

=> because proc & func are stored in db so they are secured and only
authorized users can execute the programs.

4 invoked from front-end :-

=> these programs can be called from front-end applications liike java/.net/python etc

5 improves performance :-

=> proc & func improves performance because of one time compilation i.e.
when we create a procedure program is compiled and stored in db
and whenever we call procedure only execution is repeated but not
compilation ,so this improves performance.

procedures :-

=> a procedure is a named PL/SQL block that accepts some input
performs some action on db and may or may not returns a value.

=> procedures are created to perform one or more dmls operations on tables

=> procedures are 2 types

1 standalone or stored procedures

2 packaged procedures

SERVER

 DATABASE

 BATCH32

 PROCEDURE

 PACKAGE

 PROCEDURE

standalone or stored procedures :-

=>these programs are called standalone because they are stored as
separate object in db

```
CREATE OR REPLACE PROCEDURE <NAME>
(
    parameters if any
)
IS
    declaration-part;
BEGIN
    execution-part;
END;
/
```

parameters :-

=> we can declare parameters and we can pass values to parameters
=> parameters are 3 types

- 1 IN (DEFAULT) => always receives value
- 2 OUT => always sends value
- 3 IN OUT => receives and sends

MAIN PROG		PROCEDURE
X	----->	A (IN)
Y	<-----	B (OUT)
Z	----->	C (IN OUT)
	<-----	

Example 1 :- without parameters

=> create a procedure to increment all the employee salaries by 1000 ?

```
CREATE OR REPLACE PROCEDURE raise_salary
IS
```

```

BEGIN
    UPDATE emp SET sal = sal + 1000 ;
    COMMIT;
END;
/

```

procedure created (compiled + stored in db)

Execution :-

- 1 sql prompt
- 2 another pl/sql prog
- 3 front-end applications

executing from sql prompt :-

EXECUTE procname(parameters);

ex :-

SQL>EXECUTE raise_salary ;

Example 2 :- procedure with parameters

=> create a procedure to increment specific employee sal by specific amount ?

```

CREATE OR REPLACE PROCEDURE raise_salary
(
    peno IN NUMBER,
    pamt IN NUMBER
)
IS
BEGIN
    UPDATE emp SET sal = sal + pamt WHERE empno = peno ;
    COMMIT;
END;
/

```

Execution :-

SQL>EXECUTE raise_salary(7369,1000); positional association

SQL>EXECUTE raise_salary(peno=>7369,pamt=>1000); named association

Example 3 :- procedure with out parameter

=> create a procedure to increment specific employee sal by specific amount
and after increment send the updated sal to calling program ?

```
CREATE OR REPLACE PROCEDURE raise_salary
(
    peno IN NUMBER,
    pamt IN NUMBER,
    pnewsal OUT NUMBER
)
IS
BEGIN
    UPDATE emp SET sal = sal + pamt WHERE empno = peno;
    COMMIT;
    SELECT sal INTO pnewsal FROM emp WHERE empno = peno;
END;
/
```

Execution :-

```
SQL>VARIABLE K NUMBER
SQL> EXECUTE RAISE_SALARY(7369,1000,:K);
SQL> PRINT :K
```

```
      K
-----
    4800
```

```
SQL> EXECUTE RAISE_SALARY(peno=>7369,pamt=>1000,pnewsal=>:K);
```

Declaring parameters with default value :-

```
-----
CREATE OR REPLACE PROCEDURE raise_salary
(
    peno IN NUMBER,
    pamt IN NUMBER DEFAULT 500,
    pnewsal OUT NUMBER
```

```

)
IS
BEGIN
    UPDATE emp SET sal = sal + pamt WHERE empno = peno;
    COMMIT;
    SELECT sal INTO pnewsal FROM emp WHERE empno = peno;
END;
/

```

NOTE :- while calling procedure if we don't pass value to parameter then oracle assigns default value.

=> when parameters declared with default value then while calling procedure use named association.

Execution :-

```

SQL>VARIABLE K NUMBER
SQL>EXECUTE raise_salary(peno=>7369,pnewsal=>:k);
SQL>PRINT :k

```

31-MAY-24

=> create procedure for money withdrawl ?

ACCOUNTS		
ACCNO	ACTYPE	BAL
100	S	10000
101	S	20000

TRANSACTIONS				
TRID	TTYPE	DATE	TAMT	ACCNO

```

CREATE SEQUENCE S10
START WITH 1
INCREMENT BY 1
MAXVALUE 99999;

```

```

CREATE OR REPLACE PROCEDURE DEBIT
(
    pacno IN NUMBER,
    pamt IN NUMBER,
    pnewbal OUT NUMBER

```

```

)
IS
    vbal accounts.bal%TYPE;
BEGIN
    SELECT bal INTO vbal FROM accounts WHERE accno = pacno;
    IF pamt > vbal THEN
        RAISE_APPLICATION_ERROR(-20001,'insufficient balance');
    END IF;
    UPDATE accounts SET bal = bal - pamt WHERE accno = pacno;
    INSERT INTO transactions VALUES(s10.nextval,'w',sysdate,pamt,pacno);
    COMMIT;
    SELECT bal INTO newbal FROM accounts WHERE accno = pacno;
END;
/

```

Execution :-

```

SQL>VARIABLE B NUMBER
SQL> EXECUTE DEBIT(100,5000,:B);
SQL> PRINT :B

```

=> create procedure for money deposit ?
=> create procedure for money transfer ?

USER DEFINED FUNCTIONS :-

=> functions created by user are called user defined functions.

=> when predefine functions not meeting our requirements
we create our own functions called user defined functions.

=> a function is also a named PL/SQL block that accepts some input
performs some calculation and must return a value.

=> functions are created

- 1 for calculations
- 2 to fetch data from table

```

CREATE OR REPLACE
    FUNCTION <name>(parameters) RETURN <type>
IS

```

```

    declaration-part;
BEGIN
    statements;
    RETURN expr;
END;
/

```

Example :-

```

CREATE OR REPLACE FUNCTION calc
(
    a NUMBER,
    b NUMBER,
    op CHAR
) RETURN NUMBER
IS
BEGIN
    IF op='+' THEN
        RETURN (a+b) ;
    ELSIF op='-' THEN
        RETURN(a-b);
    ELSIF op='*' then
        RETURN(a*b);
    ELSE
        RETURN(a/b);
    END IF;
END;
/

```

Execution :-

- 1 SQL commands
- 2 another pl/sql prog
- 3 front-end

executing from SQL commands :-

```
SQL>SELECT CALC(10,20,'*') FROM DUAL ;
```

=> create a function that accept year and returns whether it is leap year or not ?

```
CREATE OR REPLACE FUNCTION IS_LEAP
```

```

(
  y NUMBER
) RETURN VARCHAR2
IS
  d DATE;
BEGIN
  d := '29-FEB-'||y;
  RETURN 'leap year';
EXCEPTION
  WHEN OTHERS THEN
    RETURN 'not a leap year';
END;
/

```

Execution :-

SQL>SELECT IS_LEAP(2024) FROM DUAL ; => leap year

SQL>SELECT IS_LEAP(2023) FROM DUAL ; => not a leap year

=> difference between procedures & functions ?

PROCEDURES	FUNCTIONS
------------	-----------

- | | |
|--------------------------------------|----------------------------------|
| 1 may or may not returns a value | must return a value |
| 2 can return multiple values | always returns one value |
| 3 returns values using OUT parameter | returns value using return stmt |
| 4 cannot be called from sql commands | can be called from sql commands |
| 5 create procedure to perform dmls | create function for calculations |
| 6 create procedure to update balance | create function to get balance |

USER_SOURCE :-

=> stores procedures & functions created by user

NAME TYPE LINE TEXT

```

IS_LEAP FUNC 1      FUNCTION IS_LEAP(Y NUMBER)
              2      (
              3      Y NUMBER
              4      )
              5      IS
              6      D DATE;
              7      BEGIN

```

=> list of procedures & functions ?

```
SELECT DISTINCT NAME,TYPE FROM USER_SOURCE ;
```

=> display debit procedure code ?

```
SELECT TEXT FROM USER_SOURCE WHERE NAME='DEBIT' ;
```

Dropping :-

```
DROP PROCEDURE RAISE_SALARY ;
```

```
DROP FUNCTION IS_LEAP ;
```

=====

PACKAGES :-

=> a package is a collection of procedures & functions.

=> related procedures & functions are grouped into one package.

=> package contains 2 parts

1 package specification

2 package body

Advantages :-

1 easy to manage

2 supports overloading

3 supports hiding

package specification :-

=> package specification contains declarations of procedures & functions

```
CREATE OR REPLACE PACKAGE <name>
AS
  procedure declaration ;
  function declaration;
  -----
END;
/
```

package body :-

=> package body contains definitions of procedures & functions

```
CREATE OR REPLACE PACKAGE BODY <NAME>
AS
  PROCEDURE <NAME>(paramters)
  IS
  BEGIN
    -----
    END <proc-name>;
  FUNCTION <NAME>(parameters) RETURN <type>
  IS
  BEGIN
    -----
    END <fun-name>;
END;
/
```

Example 1 :-

=> create a package to implement following operations ?

- 1 hire employee (proc)
- 2 fire employee (proc)
- 3 raise salary (proc)
- 4 calculate experience (func)

package specification :-

```

CREATE OR REPLACE PACKAGE HR
AS
PROCEDURE HIRE(e NUMBER,n VARCHAR2,j VARCHAR2,s NUMBER,d NUMBER);
PROCEDURE FIRE(e NUMBER);
PROCEDURE RAISE_SALARY(E NUMBER,AMT NUMBER);
FUNCTION EXPR(E NUMBER) RETURN NUMBER;
END;
/

```

package body :-

```

-----

CREATE OR REPLACE PACKAGE BODY HR
AS
PROCEDURE HIRE(e NUMBER,n VARCHAR2,j VARCHAR2,s NUMBER,d NUMBER)
IS
BEGIN
    INSERT INTO emp(empno,ename,job,sal,deptno,hiredate)
        VALUES(e,n,j,s,d,sysdate);
    COMMIT;
END HIRE;
PROCEDURE FIRE(e NUMBER)
IS
BEGIN
    DELETE FROM EMP WHERE EMPNO = e;
    COMMIT;
END FIRE;
PROCEDURE RAISE_SALARY(E NUMBER,AMT NUMBER)
IS
BEGIN
    UPDATE EMP SET SAL = SAL + AMT WHERE EMPNO = E;
    COMMIT;
END RAISE_SALARY ;
FUNCTION EXPR(E NUMBER) RETURN NUMBER
IS
    VHIRE DATE;
    VEXPR NUMBER(2);
BEGIN
    SELECT HIREDATE INTO VHIRE FROM EMP WHERE EMPNO = E;
    VEXPR := (SYSDATE - VHIRE)/365;
    RETURN VEXPR;
END EXPR;

```

```
END;  
/
```

execution :-

```
SQL>EXECUTE HR.HIRE(333,'ABC','CLERK',4000,20);
```

```
SQL>EXECUTE HR.RAISE_SALARY(333,1000);
```

```
SQL>SELECT HR.EXPR(7844) FROM DUAL ;
```

=> create a package to implement various bank transactions ?

ACCOUNTS

ACCNO	ACTYPE	BAL
100 S	10000	
101 S	20000	

TRANSACTIONS

TRID	TTYPE	DATE	TAMT	ACCNO
------	-------	------	------	-------

```
CREATE SEQUENCE S10
```

```
START WITH 1
```

```
INCREMENT BY 1
```

```
MAXVALUE 99999;
```

- 1 account opening (proc)
- 2 account closing (proc)
- 3 balance enquiry (func)
- 4 money deposit (proc)
- 5 money withdrawl (proc)
- 6 money transfer (proc)
- 7 statement between two given dates (func)
- 8 latest N transactions of particular customer (func)

specification :-

```
CREATE OR REPLACE PACKAGE BANK
```

```
AS
```

```
PROCEDURE OPEN_ACCT(a NUMBER,t CHAR,b NUMBER);
```

```

PROCEDURE CLOSE_ACCT(a NUMBER);
FUNCTION GETBAL(a NUMBER) RETURN NUMBER;
PROCEDURE CREDIT(a NUMBER,amt NUMBER);
PROCEDURE DEBIT(a NUMBER,amt NUMBER);
PROCEDURE TRANSFER(s NUMBER,t NUMBER,amt NUMBER);
FUNCTION GETSTMT(a NUMBER,s DATE,e DATE) RETURN SYS_REFCURSOR;
FUNCTION GETSTMT(a NUMBER,n NUMBER) RETURN SYS_REFCURSOR;
END;
/

```

Body :-

```

CREATE OR REPLACE PACKAGE BODY BANK
AS
PROCEDURE OPEN_ACCT(a NUMBER,t CHAR,b NUMBER)
IS
BEGIN
    INSERT INTO accounts VALUES(a,t,b);
    COMMIT;
END OPEN_ACCT;
PROCEDURE CLOSE_ACCT(a NUMBER)
IS
BEGIN
    DELETE FROM accounts WHERE accno = a;
    COMMIT;
END CLOSE_ACCT;
FUNCTION GETBAL(a NUMBER) RETURN NUMBER
IS
    vbal NUMBER;
BEGIN
    SELECT bal INTO vbal FROM accounts WHERE accno = a ;
    RETURN vbal;
END GETBAL;
PROCEDURE CREDIT(a NUMBER,amt NUMBER)
IS
BEGIN
    UPDATE accounts SET bal = bal + amt WHERE accno = a ;
    INSERT INTO transactions VALUES(s10.nextval,'D',sysdate,amt,a);
    COMMIT;
END CREDIT;
PROCEDURE DEBIT(a NUMBER,amt NUMBER)
IS

```

```

vbal NUMBER;
BEGIN
    vbal := GETBAL(a);
    IF amt > vbal THEN
        RAISE_APPLICATION_ERROR(-20001,'insufficient balance');
    END IF;
    UPDATE accounts SET bal = bal - amt WHERE accno = a ;
    INSERT INTO transactions VALUES(s10.nextval,'W',sysdate,amt,a);
    COMMIT;
END DEBIT;
PROCEDURE TRANSFER(s NUMBER,t NUMBER,amt NUMBER)
IS
BEGIN
    DEBIT(s,amt);
    CREDIT(t,amt);
END TRANSFER;
FUNCTION GETSTMT(a NUMBER,s DATE,e DATE) RETURN SYS_REFCURSOR
IS
C1 SYS_REFCURSOR;
BEGIN
    OPEN C1 FOR SELECT *
                FROM TRANSACTIONS
                WHERE ACCNO = a
                  AND
                  TDATE BETWEEN s AND e ;

    RETURN C1;
END GETSTMT;
FUNCTION GETSTMT(a NUMBER,n NUMBER) RETURN SYS_REFCURSOR
IS
C1 SYS_REFCURSOR;
BEGIN
    OPEN C1 FOR SELECT *
                FROM (SELECT *
                      FROM TRANSACTIONS
                      WHERE ACCNO = a
                      ORDER BY TDATE DESC)
                WHERE ROWNUM <= n;

    RETURN C1;
END GETSTMT;
END;
/

```

Dropping :-

SQL>DROP PACKAGE HR ; => drops package specification & body

SQL>DROP PACKAGE BODY BANK ; => drops only package body but not specification

=====

TRIGGERS :-

=====

=> a trigger is a named PL/SQL block like procedure but executed implicitly by oracle whenever user submits dml command to oracle.

=> triggers are created

- 1 to control dmls
- 2 to enforce complex rules and validations
- 3 to audit day-to-day operations

```
CREATE OR REPLACE TRIGGER <NAME>
BEFORE / AFTER INSERT OR UPDATE OR DELETE
ON TABNAME
[FOR EACH ROW]
BEGIN
    STATEMENTS;
END;
/
```

BEFORE triggers :-

=> if trigger is before then trigger is executed before dml

AFTER triggers :-

=> if trigger is after then trigger is executed after dml

Ex :-

=> create trigger to not to allow dmls on emp on sunday ?

```

CREATE OR REPLACE TRIGGER T1
BEFORE INSERT OR UPDATE OR DELETE
ON EMP
BEGIN
    IF TO_CHAR(SYSDATE,'DY')='SUN' THEN
        RAISE_APPLICATION_ERROR(-20001,'sunday not allowed');
    END IF;
END;
/

```

=> create trigger to not to allow dmls as follows

```

MON - FRI  <10am AND >4pm
SAT        <10am AND >2pm
SUN        -----

```

```

CREATE OR REPLACE TRIGGER T2
BEFORE INSERT OR UPDATE OR DELETE
ON EMP
BEGIN
    IF TO_CHAR(SYSDATE,'D') BETWEEN 2 AND 6 THEN
        IF TO_CHAR(SYSDATE,'HH24') < 10
            OR
            TO_CHAR(SYSDATE, 'HH24') >= 16 THEN
            RAISE_APPLICATION_ERROR(-20001,'only between 10am and 4pm');
        END IF;
    ELSIF TO_CHAR(SYSDATE,'DY')='SAT' THEN
        IF TO_CHAR(SYSDATE,'HH24') < 10
            OR
            TO_CHAR(SYSDATE, 'HH24') >= 14 THEN
            RAISE_APPLICATION_ERROR(-20001,'only between 10am and 2pm');
        END IF;
    ELSE
        RAISE_APPLICATION_ERROR(-20001,'sunday not allowed');
    END IF;
END;
/

```

03-jun-24

=> create trigger to not to update empno ?

```

CREATE OR REPLACE TRIGGER T3

```

```

BEFORE UPDATE OF EMPNO ,HIREDATE
ON EMP
BEGIN
    RAISE_APPLICATION_ERROR(-20001,'empno and hiredate cannot be updated');
END;
/

```

Testing :-

SQL>UPDATE EMP SET EMPNO = 9999 WHERE EMPNO = 7566 ; => ERROR

:NEW ,:OLD variables :-

=> these two variables are called bind variables and rowtype variables.
=> record user is trying to insert is copied to :NEW variable.
=> record user is trying to delete is copied to :OLD variable.
=> record user is trying to update is copied to both :NEW & :OLD variables

INSERT INTO EMP VALUES(100,'A','CLERK',4000,----) => :NEW

DELETE FROM EMP WHERE EMPNO = 7369; => :OLD

UPDATE EMP SET SAL = 4000 WHERE EMPNO = 7839 ; => :OLD

:NEW

=> create trigger to not to allow to decrement salary ?

```

CREATE OR REPLACE TRIGGER T4
BEFORE UPDATE
ON EMP
FOR EACH ROW
BEGIN
    IF :NEW.SAL < :OLD.SAL THEN
        RAISE_APPLICATION_ERROR(-20001,'sal cannot be decremented');
    END IF;
END;
/

```

=> create trigger to insert details into emp_resign whenever employee resigns from organization ?

EMP_RESIGN
EMPNO ENAME HIREDATE JOB SAL DOR

```
CREATE TABLE EMP_RESIGN
(
  EMPNO  NUMBER(4),
  ENAME  VARCHAR2(10),
  HIREDATE DATE,
  JOB    VARCHAR2(10),
  SAL    NUMBER(7,2),
  DOR    DATE
);
```

```
CREATE OR REPLACE TRIGGER T5
AFTER DELETE
ON EMP
FOR EACH ROW
BEGIN
  INSERT INTO EMP_RESIGN
VALUES(:OLD.EMPNO,:OLD.ENAME,:OLD.HIREDATE,:OLD.JOB,:OLD.SAL,SYSDATE);
END;
/
```

Testing :-

DELETE FROM EMP WHERE EMPNO = 7369 ; => :OLD

USER_TRIGGERS :-

=> stores triggers created by user

```
SELECT TRIGGER_NAME,TRIGGER_TYPE,TRIGGERING_EVENT
FROM USER_TRIGGERS
WHERE TABLE_NAME='EMP' ;
```

Dropping :-

SQL>DROP TRIGGER T1 ;

cursors
procedures
functions
packages
triggers

=====

VIEWS :-

=> a view is a virtual table because it doesn't store data and doesn't occupy memory and it always derives data from base table.

=> a view always represents a query

CREATE VIEW <NAME>
AS
SELECT STATEMENT ;

=> with the help of view complex queries can be converted into simple queries

ex 1 :-

CREATE VIEW CV1
AS
SELECT e.empno,e.ename,e.sal,
 d.deptno,d.dname,d.loc
FROM emp e INNER JOIN dept d
 ON e.deptno = d.deptno ;

=> after creating view whenever user wants data from emp & dept tables then instead of writing join query write the simple query

SELECT * FROM CV1 ;

ex 2 :-

```
CREATE VIEW CV2
AS
SELECT d.dname,sum(e.sal) as totalsal
FROM emp e INNER JOIN dept d
      ON e.deptno = d.deptno
GROUP BY d.dname ;
```

=> after creating view whenever we want dept wise summary then execute the following query

```
SELECT * FROM CV2 ;
```

SET OPERATORS :-

```
UNION
UNION ALL
INTERSECT
MINUS
```

```
SELECT STATEMENTS 1
UNION / UNION ALL / INTERSECT / MINUS
SELECT STATEMENT 2 ;
```

```
SELECT JOB FROM EMP WHERE DEPTNO = 20 ;
```

```
CLERK
MANAGER
ANALYST
CLERK
ANALYST
```

```
SELECT JOB FROM EMP WHERE DEPTNO = 30 ;
```

```
JOB
```

```
SALESMAN
SALESMAN
SALESMAN
MANAGER
SALESMAN
CLERK
```

UNION :-

=> combines rows return by two queries
=> duplicates are eliminated
=> result is sorted

```
SELECT JOB FROM EMP WHERE DEPTNO = 20  
UNION  
SELECT JOB FROM EMP WHERE DEPTNO = 30 ;
```

ANALYST
CLERK
MANAGER
SALESMAN

UNION ALL :-

=> combines rows return by two queries
=> duplicates are not eliminated
=> result is not sorted

```
SELECT JOB FROM EMP WHERE DEPTNO = 20  
UNION ALL  
SELECT JOB FROM EMP WHERE DEPTNO = 30 ;
```

CLERK
MANAGER
ANALYST
CLERK
ANALYST
SALESMAN
SALESMAN
SALESMAN
MANAGER
SALESMAN
CLERK

INTERSECT :-

=> returns common values from the output of two select statements

```
SELECT JOB FROM EMP WHERE DEPTNO = 20  
INTERSECT  
SELECT JOB FROM EMP WHERE DEPTNO = 30 ;
```

CLERK
MANAGER

MINUS :-

=> returns values present in 1st query output and not present in 2nd query output

```
SELECT JOB FROM EMP WHERE DEPTNO = 20  
MINUS  
SELECT JOB FROM EMP WHERE DEPTNO = 30 ;
```

CLERK

Question :-

T1	T2
F1	C1
1	1
2	2
3	3
10	40
20	50
30	60

write outputs for the following operations ?

- 1 INNER JOIN
- 2 LEFT OUTER JOIN
- 3 RIGHT OUTER JOIN
- 4 FULL OUTER JOIN
- 5 UNION
- 6 UNION ALL
- 7 INTERSECT
- 8 MINUS

EMP_US

ENO ENAME SAL DNO

				DEPT		
EMP_IND				DNO	DNAME	LOC
ENO	ENAME	SAL	DNO			

=> total employee list ?

```
SELECT * FROM EMP_US
UNION
SELECT * FROM EMP_IND ;
```

=> employees working at us loc with dept details ?

```
SELECT E.*,D.*
FROM EMP_US E INNER JOIN DEPT D
ON E.DNO = D.DNO ;
```

=> total employees with dept details ?

```
SELECT E.*,D.*
FROM EMP_US E INNER JOIN DEPT D
ON E.DNO = D.DNO
UNION
SELECT E.*,D.*
FROM EMP_IND E INNER JOIN DEPT D
ON E.DNO = D.DNO ;
```

```
SELECT E.*,D.*
FROM (SELECT * FROM EMP_US
UNION
SELECT * FROM EMP_IND) E INNER JOIN DEPT D
ON E.DNO = D.DNO ;
```


